

华中科技大学

研究生（文献阅读与选题）报告

☐文献阅读、☐选题、☐文献阅读与选题、☐论文中期进展

题目：AI Agent 运行轨迹与机制关联的交互设计研究

学	号	<u>M202570493</u>
姓	名	<u>陈思静</u>
专	业	<u>设计学</u>
指 导 教 师		<u>何心</u>
院（系、所）		<u>设计学院</u>

填表注意事项

- 一、本表适用于攻读硕士、博士学位研究生文献阅读报告、选题报告、文献阅读与选题报告、论文中期进展报告。
- 二、以上各报告内容及要求由相关院（系、所）做具体要求。
- 三、以上各报告均须存入研究生个人学籍档案。
- 四、本表填写要求文句通顺、内容明确、字迹工整。
- 五、A4 纸正反打印

□文献阅读、□选题、□文献阅读与选题、□论文中期进展

目录

一、文献阅读卡

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：01

名称 Results-Actionability Gap: Understanding How Practitioners Evaluate LLM Products in the Wild

作者 Willem van der Maden; Malak Sadek; Ziang Xiao; Aske Mottelson; Q. Vera Liao; Jichen Zhu 译者

出处 【美国】Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, 2026: 1-17 页

文献阅读心得 (每篇不少于 150 字):

研究背景 LLM 产品的评价对象不是孤立模型，而是包含提示词、检索、界面与业务流程的完整系统。传统基准分数很难直接回答产品团队下一步应改什么，实践者因此在不确定性、资源限制和多方目标之间组织评价工作。

研究问题 RQ1: 生产环境中的 LLM 产品目前采用哪些评价实践?

RQ2: 实践者认为评价 LLM 产品时最主要的困难是什么?

研究方法 作者对 19 名跨行业 LLM 产品实践者开展半结构化访谈，并进行主题分析。研究覆盖评价执行、方案设计及组织层面的协调工作，分析对象是面向真实用户的完整产品。

主要发现 研究归纳出 10 类评价活动和 5 类挑战。17 名参与者经历了“结果-可行动性鸿沟”：团队能收集反馈或分数，却难以判断失败来自提示、检索、模型还是其他部件。人工判断与“vibe check”被视为对 LLM 特性的适应，而不只是方法不成熟。

贡献与意义 论文提出“formalization journey”和“results-actionability gap”，把评价后的定位、协商与改进纳入评价研究，并从较成熟团队中总结了可实施的组织与流程策略。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：02

名称 Interactive Debugging and Steering of Multi-Agent AI Systems

作者 Will Epperson; Gagan Bansal; Victor Dibia; Adam Fourney; Jack Gerrits; Erkang Zhu; Saleema Amershi 译者

出处 【美国】Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, 2025: 156:1–156:15 页

文献阅读心得 (每篇不少于 150 字):

研究背景 多 Agent 系统会产生数十甚至上百条消息，错误可能在早期计划、角色分工或工具调用中出现，并沿后续协作传播。现有开发工具多支持事后阅读日志，缺少暂停、回退和试改轨迹的交互能力。

研究问题

RQ1: 开发者在构建和调试多 Agent 工作流时面临哪些核心困难?

RQ2: 消息级浏览、编辑与重置如何支持故障诊断和 Agent steering?

研究方法 研究先访谈 5 名 Agent 开发者，据此设计 AGDebugger; 随后开展含 14 名参与者的两阶段用户研究。第一阶段比较 AGDebugger 与基线日志界面的错误识别，第二阶段让参与者编辑历史消息并重新运行。

主要发现 参与者通常花约 10 分钟阅读日志后才开始修改。5/6 的第一阶段参与者偏好 AGDebugger; 第二阶段 8 人共做 24 次编辑，其中 14 次是补充具体指令，另有简化指令和修改计划。重置与编辑功能评分最高，但只有 2/8 得到完全正确答案。

贡献与意义 AGDebugger 把历史概览、逐条消息检查、回退、编辑与继续执行放进同一界面，使开发者能够用反事实式试验观察某次干预是否改变后续轨迹。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：03

名称 Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems

作者 Shaokun Zhang; Ming Yin; Jieyu Zhang; Jiale Liu; Zhiguang Han; Jingyang Zhang; Beibin Li; Chi Wang; Huazheng Wang; Yiran Chen; Qingyun Wu 译者

出处 【美国】Proceedings of Machine Learning Research, 2025, 267: 76583–76599 页

文献阅读心得 (每篇不少于 150 字):

研究背景 多 Agent 任务失败时，最后输出错误的 Agent 不一定是决定结果的源头。有效调试需要同时定位责任 Agent 和关键步骤，但长轨迹、角色交互和错误传播使人工归因成本很高。

研究问题

RQ1: 能否自动识别导致任务失败的 Agent，即回答“谁”造成失败？

RQ2: 能否定位该 Agent 产生决定性错误的具体步骤，即回答“何时”出错？

研究方法 作者将修正某个 Agent—时间对后能否把失败变为成功作为决定性错误的反事实定义，并以最早的决定性错误为标注目标。Who&When 数据集包含 127 个多 Agent 系统、184 个失败归因任务，比较 all-at-once、binary search 和 step-by-step 三类方法。

主要发现 最佳方法的 Agent 级准确率为 53.5%，步骤级准确率仅 14.2%，部分方法低于随机基线；包括先进推理模型在内的方法仍未达到实用水平。自动判断“谁”已有有限信号，而精确判断“何时”更困难。

贡献与意义 论文首次系统化多 Agent 自动故障归因，提供同时含责任 Agent、决定性步骤和自然语言原因的细粒度数据集，并把 Agent 级与步骤级能力分开评价。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：04

名称 Finding AI's Faults with AAR/AI: An Empirical Study

作者 Roli Khanna; Jonathan Dodge; Andrew Anderson; Rupika Dikkala; Jed Irvine;
Zeyad Shureih; Kin-Ho Lam; Caleb R. Matthews; Zhengxian Lin; Minsuk Kahng;
Alan Fern; Margaret Burnett 译者

出处 【美国】ACM Transactions on Interactive Intelligent Systems, 2022,
12(1): 1:1–1:33 页

文献阅读心得（每篇不少于 150 字）：

研究背景 总体成败或平均性能无法告诉领域用户 AI 究竟在哪些情境下不可靠。要决定何时依赖 Agent，用户需要把问题定位到具体决策和内部预测，而不只是给出笼统的通过或失败判断。

研究问题

RQ1: AAR/AI 流程能否帮助具备领域知识的用户定位 AI 故障？

RQ2: 故障类型不同，用户发现故障的效果是否也不同？

研究方法 65 名有即时战略游戏经验但无 AI/ML 背景的参与者被随机分到 AAR/AI 组和对照组，分析强化学习 Agent 完成 StarCraft II Tug-of-War 任务的过程。研究用 10 个经验证的自然故障实例计算问题报告的召回率和精确率。

主要发现 AAR/AI 组召回率为 0.233，对照组为 0.080；精确率分别为 0.179 和 0.116，差异均显著。AAR/AI 参与者在每类故障上表现更好，发现任一特定故障的可能性平均接近对照组的 6 倍。

贡献与意义 论文用可核对的故障真值、召回率和精确率评价人类诊断，而非只测满意度；结构化复盘和标签活动还可能帮助用户从单个实例上升到故障类别。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：05

名称 Debugging Reinvented: Asking and Answering Why and Why Not Questions
about Program Behavior

作者 Andrew J. Ko; Brad A. Myers 译者

出处 【美国】Proceedings of the 30th International Conference on Software
Engineering, 2008: 301–310 页

文献阅读心得（每篇不少于 150 字）：

研究背景 开发者观察到异常输出后，传统调试器仍要求他们先猜测相关代码，再用断点和单步执行验证。由于初始猜测经常错误，问题表述与工具操作之间的转换会产生大量无关搜索。

研究问题

RQ1: 如何从程序代码和一次执行中自动提取用户可能提出的 why 与 why-not 问题？

RQ2: 如何用静态、动态分析回答这些问题，并把答案映射到相关运行事件和代码？

研究方法 论文给出 Java Whyline 的系统设计与算法：记录完整执行历史，从文本和图形输出生成问题，结合动态切片、静态切片、调用图、可达性分析和值来源分析构造因果解释，并讨论性能与覆盖限制。

主要发现 “为什么发生”可沿已执行依赖回溯；“为什么没有发生”必须分析潜在定义、未满足条件与未到达路径，计算和问题空间更复杂。原型证明可以从可见输出进入相关代码，而无需先猜故障位置。

贡献与意义 文章把调试问题作为一等交互对象，并提出面向大型 Java 程序自动生成、筛选和回答 why/why-not 问题的完整技术架构。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：06

名称 Designing the Whyline: A Debugging Interface for Asking Questions about Program Behavior

作者 Andrew J. Ko; Brad A. Myers 译者

出处 【美国】Proceedings of the 2004 ACM Conference on Human Factors in Computing Systems, 2004: 151–158 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试本质上从对运行行为的疑问开始，但断点、单步和打印语句迫使程序员先定位代码。Whyline 尝试让用户直接针对输出提问，再由系统呈现与该问题有关的运行时因果链。

研究问题

RQ1: 用户是否认为基于 why/why-not 提问的 Whyline 有用？

RQ2: Whyline 能否缩短调试时间并帮助程序员完成更多任务？

研究方法 作者在 Alice 编程环境中实现 Whyline，并对既有无 Whyline 研究和新增有 Whyline 研究中的相同场景进行比较。无工具组 4 人、有工具组 5 人，在 90 分钟内完成 6 项 Pac-Man 编程任务并出声思考。

主要发现 在 6 个可比调试场景中，Whyline 把调试时间平均缩短 7.8 倍；有工具组平均完成 3.20 项任务，无工具组为 2.25 项，提升约 40%。19/24 次提问中工具被认为有用，why-not 问题又明显多于 why 问题。

贡献与意义 Whyline 把可见程序输出转成问题菜单，并用时间游标、因果箭头和可逆历史把现象、运行事件与代码连接起来，改变了从代码位置开始的调试顺序。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：07

名称 Finding Causes of Program Output with the Java Whyline

作者 Andrew J. Ko; Brad A. Myers 译者

出处 【美国】Proceedings of the 27th ACM Conference on Human Factors in Computing Systems, 2009: 1569–1578 页

文献阅读心得（每篇不少于 150 字）：

研究背景 大型 Java 程序中，断点调试要求开发者先选对代码位置，而用户通常更容易识别界面中出现或缺失的输出。Java Whyline 把“从结果反查原因”的思路扩展到真实开源项目。

研究问题

RQ1: Java Whyline 是否比断点调试器更快、更成功地帮助开发者定位真实缺陷？

RQ2: 两种工具如何改变开发者浏览文件、使用问题和接近关键函数的路径？

研究方法 研究采用组间实验，两组各 10 名软件工程硕士生，分别使用 Whyline 或具有相同外观的断点调试器，诊断 ArgoUML 的两个真实缺陷。每项任务限时 30 分钟，并记录成功率、时间、文件访问和导航距离。

主要发现 任务 1 中 Whyline 组 10/10 成功，对照组 3/10，且前者约快 2 倍；任务 2 中分别为 4/10 和 0/10。总体上 Whyline 用户约有 3 倍成功率，并更快接近关键代码；但工具更善于找到故障方法，不一定能解释方法为何错误。

贡献与意义 研究把输出选择、执行录像、依赖可视化、书签与问题导航集成为可实证比较的调试界面，并使用真实大型项目而非人工小程序。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：08

名称 How Programmers Debug, Revisited: An Information Foraging Theory
Perspective

作者 Joseph Lawrance; Christopher Bogart; Margaret Burnett; Rachel Bellamy;
Kyle Rector; Scott D. Fleming 译者

出处 【美国】IEEE Transactions on Software Engineering, 2013, 39(2): 197–
215 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试大型代码库时，程序员不断判断下一条线索是否值得追踪。仅用“提出并验证假设”描述这一过程，很难解释他们在文件、方法和调用关系之间的实际导航，也难为工具设计提供直接建议。

研究问题

RQ1：专业程序员调试时的语言和导航是否主要体现信息气味追踪，而非显式假设？

RQ2：信息气味与代码拓扑模型能否预测程序员下一步会访问哪里？

研究方法 作者提出基于信息觅食理论的调试模型，并观察 10 名专业程序员调试真实开源程序，分析出声思考与导航日志；随后构建可执行模型，与不考虑信息气味的替代模型比较预测能力。

主要发现 参与者的言语更多涉及寻找和比较线索，而不是陈述完整假设。错误信息、标识符、文件名和调用关系形成信息气味；纳入气味的模型更准确地预测导航，代码拓扑则决定线索间移动成本。

贡献与意义 论文以 scent 和 topology 两个可观测、可计算的构念解释调试导航，减少对难以直接测量的心理假设的依赖，并把理论转成可执行预测模型。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：09

名称 Mental Procedures in Real-Life Tasks: A Case Study of Electronic Trouble Shooting

作者 Jens Rasmussen; A. Jensen 译者

出处 【英国】Ergonomics, 1974, 17(3): 293-307 页

文献阅读心得（每篇不少于 150 字）：

研究背景 熟练维修人员面对电子设备故障时，不会穷举所有测量，而是在系统、功能单元和部件组成的层级结构中搜索。真实工作还要求兼顾诊断时间、认知负荷和测量成本。

研究问题

RQ1: 熟练电子维修人员如何把设备表示为层级系统并组织故障搜索？

RQ2: 不同搜索例程在观察数量、知识深度和心理加工复杂度上有何差异？

研究方法 研究在维修人员的正常工作环境中收集口述协议，对真实电子排故过程进行细致编码，分析症状判断、层级切换、测量选择及反复出现的搜索例程。

主要发现 排故可分解为多种循环搜索例程，分别依赖不同深度的内部机理知识。维修人员偏好快速连续的好/坏判断，即使某些观察在信息论上冗余；若目标是减少时间和心理负担，这种做法仍然合理。

贡献与意义 论文较早把真实排故中的认知过程描述为层级系统上的搜索，并揭示“信息最省”与“时间、心智成本最优”并不相同。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：10

名称 Making Sense of Sensemaking 1: Alternative Perspectives

作者 Gary Klein; Brian Moon; Robert R. Hoffman 译者

出处 【美国】IEEE Intelligent Systems, 2006, 21(4): 70–73 页

文献阅读心得（每篇不少于 150 字）：

研究背景 sensemaking 在心理学、组织研究、自然情境决策和 HCI 中有多种含义。复杂任务中的理解不是对信息做一次摘要，而是持续寻找联系、解释意外并为下一步行动建立可用认识。

研究问题

RQ1: 不同研究传统如何界定 sensemaking，它与理解、情境意识等概念有何关系？

RQ2: 哪种定义能解释人在不确定环境中为了预测和行动而持续组织联系？

研究方法 本文是概念综述与立场论文。作者比较多个学科的定义和代表性案例，讨论 sensemaking 的触发条件、目的与分析尺度，并为后续宏认知模型建立概念边界。

主要发现 作者强调 sensemaking 是一种有动机、持续的努力：理解人、地点和事件之间的联系，以预测其发展并有效行动。它常由惊讶、冲突或知识缺口触发，结果是可行动的解释，而非静态信息表征。

贡献与意义 论文跨学科澄清了 sensemaking 的不同视角，把“联系—预测—行动”作为共同核心，为研究真实复杂工作中的理解活动提供了统一入口。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：11

名称 Making Sense of Sensemaking 2: A Macrocognitive Model

作者 Gary Klein; Brian Moon; Robert R. Hoffman 译者

出处 【美国】IEEE Intelligent Systems, 2006, 21(5): 88-92 页

文献阅读心得（每篇不少于 150 字）：

研究背景 人在不确定任务中会借助已有框架筛选数据，数据又可能迫使框架改变。把意义建构理解成固定步骤，会忽略专家在解释、注意分配、反馈和行动之间的往返。

研究问题

RQ1: 数据与解释框架如何相互依存并推动意义建构？

RQ2: 人何时会扩展、质疑、比较、保留或重构当前框架？

研究方法 作者基于自然情境决策研究提出 Data/Frame 宏认知模型，并用因果推理、假设承诺、反馈学习、专家差异和确认偏差等经验研究检验模型含义。

主要发现 框架管理注意并定义、连接和过滤数据；异常与不一致可触发质疑或重构。过早形成框架并非必然有害，适度承诺能产生可检验预期；过程反馈通常比只告知成败的结果反馈更有助于改进。

贡献与意义 论文以双向的 Data/Frame 循环替代线性“先收集数据、后得出解释”，并提出连接、扩展、质疑、比较、重构和寻找框架等可观察活动。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：12

名称 A Data-Frame Theory of Sensemaking

作者 Gary Klein; Jennifer K. Phillips; Erica L. Rall; Deborah A. Peluso 译者

出处 【美国】Expertise Out of Context, Mahwah, NJ: Lawrence Erlbaum Associates, 2007: 113-155 页

文献阅读心得（每篇不少于 150 字）：

研究背景 传统微观认知研究多解释人如何理解单词、句子等简单刺激，难以覆盖真实工作中对模糊、矛盾、动态信息形成可行动理解的过程。Data-Frame 理论由此转向自然情境中的宏观意义建构。

研究问题

RQ1：数据如何被框架选择和解释，框架又如何由少量关键锚点被识别或建构？

RQ2：当数据与框架不一致时，意义建构有哪些可能形式与转换路径？

研究方法 本章综合自然情境决策、专业知识和问题解决研究，借助真实事件材料提出 9 条理论命题，并系统描述连接数据与框架、扩展、质疑、保留、比较、重构和寻找新框架等活动。

主要发现 数据不是完全中性的输入，而是在框架中被辨认和赋义；框架也由关键线索推断。意义建构通常在二者达到足够一致时停止，并服务于“接下来做什么”的功能性理解。专家与新手主要差在可调用框架的丰富度。

贡献与意义 理论同时说明数据驱动和框架驱动过程，明确列出多种非线性活动及其触发关系，可用于分析解释如何被建立、维护和推翻。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：13

名称 AgentGUI: An Interface for Observing and Steering Long-Running AI Agents

作者 Xuan Zhao; Jiwoong Sohn; Qinyue Zheng; Michael Moor 译者

出处 【瑞士】arXiv preprint arXiv:2607.26300, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 长时 Agent 可同时执行多个任务，但人类监督仍依赖终端输出和分散日志。用户需要在不中断整体工作的情况下掌握轨迹、产物、成本和偏移，并在必要时实施人工或自动纠偏。

研究问题

RQ1: 统一的轨迹可视化能否提高用户理解长时 Agent 执行的速度和准确性？

RQ2: 手动与自动管理者式 steering 能否在任务漂移时改善最终完成率？

研究方法 系统整合多会话轨迹、工作区快照、人工输入和自动 Manager。8 名硕博参与者在被试内、顺序平衡实验中比较 AgentGUI 与 Hermes Dashboard；另对多模型各运行 50 次测试自动纠偏。

主要发现 参与者用 AgentGUI 回答轨迹问题平均快 38%（90 秒对 145 秒， $p=.023$ ），准确率为 93%对 80%（ $p=.031$ ），且没有速度—准确率交换；自动 Manager 在部分模型上将完成率提高最多 34 个百分点。

贡献与意义 论文把并行会话、轨迹理解、工作区产物、人工指令和自动审计放进同一桌面式界面，并用客观理解速度、准确率和 workload 评价可观测性。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：14

名称 An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks

作者 Amy J. Ko; Brad A. Myers; Michael J. Coblenz; Htet Htet Aung 译者

出处 【美国】IEEE Transactions on Software Engineering, 2006, 32(12): 971–987, DOI: 10.1109/TSE.2006.116 页

文献阅读心得（每篇不少于 150 字）：

研究背景 软件维护不仅需要读懂局部代码，还要在多文件、多工具和多任务之间寻找、关联并暂存相关信息。现代 IDE 的导航机制可能既提供线索，也制造误导与重复切换。

研究问题

RQ1：开发者在维护任务中如何搜索、关联和收集相关代码与运行信息？

RQ2：IDE 的线索、依赖导航和交互状态如何支持或妨碍这些活动？

研究方法 研究从 31 名 Java 开发者中选取经验最高的 10 名，要求其在 70 分钟内完成同一 503 行 Java Paint 程序的 5 个维护任务，并接受定时中断。作者逐帧分析屏幕录像、语音和交互日志。

主要发现 开发者平均 35% 的有效时间用于源文件内外的导航，许多跳转是重复且本可避免的；失败搜索常由误导性信息气味开始。记忆不可靠、纸笔缓慢、IDE 交互状态易被后续任务覆盖，均难稳定保存任务上下文。

贡献与意义 论文提出“搜索—关联—收集”的程序理解模型，将信息觅食与 IDE 具体交互结合，并以真实多任务和中断条件揭示导航成本及外部记忆需求。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：15

名称 Debugging: An Analysis of Bug-Location Strategies

作者 Irvin R. Katz; John R. Anderson 译者

出处 【美国】Human-Computer Interaction, 1987-1988, 3(4): 351-399,

DOI: 10.1207/S15327051HCI0304_2 页

文献阅读心得（每篇不少于 150 字）：

研究背景 具备语言知识的程序员往往能在看到错误行后修正它，真正困难的是从程序行为中定位哪一行有错。调试策略可能因代码归属、可用线索和搜索顺序而改变。

研究问题

RQ1: 熟悉 LISP 的学生在定位程序错误时采用哪些策略，错误更接近知识缺陷还是偶发失误？

RQ2: 调试自己的程序与他人程序是否改变定位策略，策略又如何影响检查与修复？

研究方法 论文报告 4 项 LISP 调试实验，比较错误重复性、自然使用的定位策略、自编与他编程序情境，以及不同策略对被检查代码行和后续判断修复的影响。

主要发现 参与者的错误多为不易重复的 slip，通常一旦定位即可修复；他们会使用多种错误定位策略，且自编与他编程序的策略选择不同。策略主要改变搜索哪些代码行，对选中一行后的正确性判断和修复能力影响较小。

贡献与意义 研究把“找错位置”与“判断并修复错误”分解为不同阶段，并通过连续实验说明调试瓶颈并非总是缺少编程知识，而是搜索与定位。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：16

名称 Theory of Troubleshooting: The Developer's Cognitive Experience of Overcoming Confusion

作者 Arty Starr; Margaret-Anne Storey 译者

出处 【美国】ACM Transactions on Software Engineering and Methodology, 2026, DOI: 10.1145/3800945; arXiv:2602.10540 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试研究常聚焦工具和步骤，却较少解释开发者面对意外行为时的内部体验。长时间困惑会持续占用注意、工作记忆和心理模型构建资源，并可能转化为疲劳与项目可持续性风险。

研究问题

RQ1：开发者在排查意外系统行为时，思考、感受和努力随时间如何变化？

RQ2：困惑、经验直觉、试探性实验、进展与认知疲劳之间存在怎样的关系？

研究方法 作者访谈 27 名职业开发者，合计 570 年经验，并开展 7 次回访；采用建构主义扎根理论，从 681 条扎根编码形成 8 个核心理论类别，在前 12 份访谈后达到理论饱和，并以参与者反馈检验共鸣。

主要发现 排故从困惑体验启动，经验直觉引导策略，开发者通过“试一试、看结果”缩小解释空间；获得正或负知识都可被体验为进展。持续无进展会耗竭注意并引发补偿努力、盲视和疲劳，最终在“想通了”时解除。

贡献与意义 论文把排故定义为识别、理解并建立意外行为原因心理模型的认知过程，用连贯理论连接情绪、注意、实验、直觉和疲劳，而不只描述技术操作。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：17

名称 The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis

作者 Peter Pirolli; Stuart K. Card 译者

出处 【美国】Proceedings of the 2005 International Conference on Intelligence Analysis, 2005 页

文献阅读心得（每篇不少于 150 字）：

研究背景 情报分析需要从大量异质材料形成结构、假设和可交付结论，但规范性流程很少说明专家真实工作中如何往返搜索、筛选、重组和验证证据。

研究问题

RQ1: 专家分析者如何在信息觅食与意义建构之间循环，将信息转化为模式、洞见和产品？

RQ2: 认知任务分析揭示了哪些可由交互技术降低成本的关键杠杆点？

研究方法 作者采用认知任务分析、分析者访谈和出声思考协议，结合专家图式与信息觅食理论，构建从外部数据源到证据文件、图式、假设和最终报告的双环模型。

主要发现 分析并非线性阅读，而是在探索、富集、利用与表示重构之间机会主义往返；常见外部结构包括 shoebox、evidence file 和 schema。高成本环节集中在搜索、筛选、重新表示、比较假设和组装证据。

贡献与意义 论文将 foraging loop 与 sensemaking loop 连接起来，并从每个转换的时间和注意成本推导工具设计杠杆，为总览—筛选—证据—假设界面提供理论基础。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：18

名称 AgentStepper: Interactive Debugging of Software Development Agents

作者 Robert Hutter; Michael Pradel 译者

出处 【德国】arXiv preprint arXiv:2602.06593, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 软件开发 Agent 会在模型请求、工具调用和代码修改之间形成长轨迹。原始日志虽然保留了大量信息，却缺少适合开发者理解高层行为的结构，也不能像传统调试器那样暂停、单步和检查中间状态。

研究问题

RQ1: 将传统调试中的断点、单步执行和状态检查提升到 Agent 动作层，能否帮助开发者理解 Agent 轨迹？

RQ2: AgentStepper 接入现有软件开发 Agent 需要多少改动，它是否改善轨迹理解、Agent 程序缺陷识别和主观负荷？

研究方法 作者把 AgentStepper 接入 ExecutionAgent、SWE-Agent 和 RepairAgent，统计集成改动量；随后邀请 12 名参与者完成一项轨迹理解任务和两项 Agent 调试任务，比较 AgentStepper 与自行选择的常规日志工具。

主要发现 三种 Agent 各需插入 5—7 个 API 调用并修改 39—42 行代码。AgentStepper 条件下轨迹理解平均表现由 64% 升至 67%，Agent 程序缺陷识别成功率由 17% 升至 60%，挫败感由 5.4/7 降至 2.4/7。

贡献与意义 论文把 Agent 轨迹表示为模型—Agent 程序和 Agent 程序—工具两条交织会话，并整合总览、细节、代码变更和断点控制。它是 V1 最直接的人侧调试先例，也说明“理解轨迹”和“定位 Harness 实现缺陷”应分开测量。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：19

名称 AgentSight: System-Level Observability for AI Agents Using eBPF

作者 Yusheng Zheng; Yanpeng Hu; Tong Yu; Andi Quinn 译者

出处 【美国】arXiv preprint arXiv:2508.02736, 2025 页

文献阅读心得（每篇不少于 150 字）：

研究背景 Agent 的高层意图存在于模型提示和响应中，实际效果却表现为进程、文件和网络事件。现有工具通常只能看到其中一侧，导致开发者无法判断某个系统行为是否由 Agent 意图、工具实现或外部环境造成。

研究问题

RQ1: 能否在不修改 Agent 框架代码的情况下关联模型层语义意图与系统层实际行为？

RQ2: 这种跨层观测能否以可接受的开销识别安全攻击、低效循环和多 Agent 协调问题？

研究方法 作者提出 boundary tracing，利用 eBPF 监视内核事件，同时截获 TLS 加密的模型流量提取语义意图，再通过实时关联引擎和辅助 LLM 跨进程连接两类事件，并在多种攻击和故障场景中评估。

主要发现 AgentSight 能够检测提示注入、识别浪费资源的推理循环并发现隐藏的多 Agent 协调瓶颈，运行开销低于 3%。研究也表明，系统级事实和语义解释来自不同观测通道。

贡献与意义 论文具体展示了跨层证据如何被关联，并揭示“直接记录的系统事件”和“经过模型分析得到的语义关系”不能混为同一种证据。它为 V1 的记录到、推导和未知三类证据状态提供了技术边界。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：20

名称 From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws

作者 Mengzhuo Chen; Junjie Wang; Zhe Liu; Yawen Wang; Haiming Zheng; Qing Wang 译者

出处 【中国】arXiv preprint arXiv:2606.06324v2, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 Agent 失败往往由执行环境、工具、上下文、生命周期、可观测、验证或治理机制共同塑造。仅依据最终成败调整提示词，难以确定失败证据位于哪一步，也难以把运行现象映射到具体 Harness 工件。

研究问题

RQ1：如何把碎片化运行轨迹编译成能够表达步骤级数据流、控制流和 Harness 实现锚点的统一表示？

RQ2：基于该表示形成的局部诊断和范围化修复，能否在不引入不可接受回归的情况下改善 Agent 表现？

研究方法 HarnessFix 将原始轨迹与 Harness 工件编译为 HTIR，定位责任步骤和实现工件，把重复诊断合并为缺陷记录，再按缺陷类型生成补丁并做回归约束验证；实验覆盖四个公开基准。

主要发现 相较初始 Harness，HarnessFix 在四个基准上的性能提升为 6.3%—18.4%，并优于人工设计和自演化基线。结果显示，失败轨迹可被组织成指向具体 Harness 机制的结构化证据。

贡献与意义 论文证明数据流、控制流和实现锚点可以同时进入 Harness 感知的轨迹表示，是 V1 最接近的技术工作。其自动修复不是 V1 目标，但 HTIR 所物化的关联类型构成了人侧理解界面的关键比较对象。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：21

名称 Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses

作者 Jiahang Lin; Shichun Liu; Chengjun Pan; Lizhi Lin; Shihan Dou; Zhiheng Xi; Xuanjing Huang; Hang Yan; Zhenhua Han; Tao Gui; Yu-Gang Jiang 译者

出处 【中国】arXiv preprint arXiv:2604.25850v4, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 Harness 包含工具、中间件、长期记忆、系统提示和运行规则，修改空间异构，轨迹规模巨大，而且一次修改与下一轮结果之间的关系难以归因。自动演化若缺少可观测证据，容易退化为试错。

研究问题

RQ1: 组件、运行经验和修改决策需要怎样的可观测表示，才能形成可验证的 Harness 演化闭环？

RQ2: 可观测性驱动自动演化能否提高编码 Agent 性能，并跨模型迁移而非只适配单一基准？

研究方法 作者提出组件可观测、经验可观测和决策可观测三支柱，在 Terminal-Bench 2 上执行 10 轮 Harness 演化，并通过消融和跨模型、跨基准迁移检验各类组件修改的作用。

主要发现 Terminal-Bench 2 的 pass@1 由 69.7% 升至 77.0%；冻结后的 Harness 在其他模型家族上带来 5.1—10.1 个百分点提升，并以比种子 Harness 少 12% 的 token 取得 SWE-bench Verified 上的领先结果。增益主要来自工具、中间件和长期记忆，而非系统提示。

贡献与意义 论文把可观测性从日志采集提升为组件、经验和决策三个层次，并要求每次修改都有可证伪预测。它说明 V1 不能只研究提示词，而应将上下文和工具等机制放入统一 Harness 结构。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：22

名称 AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems

作者 Jiaying Lu; Bo Pan; Jieyi Chen; Yingchaojie Feng; Jingyuan Hu; Yuchen Peng; Wei Chen 译者

出处 【美国】IEEE Transactions on Visualization and Computer Graphics, 2025, 31(8): 4182–4197, DOI: 10.1109/TVCG.2024.3394053 页

文献阅读心得（每篇不少于 150 字）：

研究背景 多 Agent 自治系统产生大量随时间演化的事件、记忆和交互。线性日志难以呈现跨时间行为模式，也难以让用户从总体演化下钻到单个 Agent 行为及其可能原因。

研究问题

RQ1: 如何把原始执行事件组织成支持多粒度查看的 Agent 行为模型，并追踪行为之间的潜在因果关系？

RQ2: 层级时间视图、单 Agent 细节视图和环境监视视图能否支持用户发现并解释复杂行为？

研究方法 作者建立事件处理、行为摘要和原因追踪流程，开发含 Outline、Agent 和 Monitor 三类联动视图的 AgentLens，并通过两个使用场景和用户研究评价系统。

主要发现 参与者能够沿层级时间线发现行为模式、下钻检查事件，并回看可能影响当前行为的历史操作。系统有效性和可用性得到验证，但部分因果关系依据文本相似度推断，并非直接观测事实。

贡献与意义 论文提供总览—细节联动和原因追踪的成熟视觉设计，也暴露了 V1 要解决的风险：界面若不说明连线来自日志、算法推断还是缺失信号，使用者可能把潜在关系理解为实际因果。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：23

名称 AgentGraph: Trace-to-Graph Platform for Interactive Analysis and Robustness Testing in Agentic AI Systems

作者 Zekun Wu; Seonglae Cho; Cristian Enrique Munoz Villalobos; Theo King; Umar Mohammed; Emre Kazim; Maria Perez-Ortiz; Sahan Bulathwela; Adriano Koshiyama 译者

出处 【美国】Proceedings of the AAAI Conference on Artificial Intelligence, 2026, 40(48): 41721–41723, DOI: 10.1609/aaai.v40i48.42393 页

文献阅读心得（每篇不少于 150 字）：

研究背景 现有 Agent 可观测平台多记录提示输入输出和运行指标，却仍要求用户手工从线性轨迹恢复任务、工具、数据和人员之间的结构关系。

研究问题

RQ1: 如何把 Agent 执行日志转换为带类型节点和边的交互知识图，并保持每个图元素可回到原始轨迹？

RQ2: 统一图表示能否同时支持故障分析、优化建议和扰动条件下的稳健性测试？

研究方法 AgentGraph 把 Agent、任务、工具、数据输入输出和人建模为节点，用消费、委托、顺序、工具使用、输出交付和人工干预等类型边连接，并让每个元素指向精确 trace span；系统还加入扰动和因果归因分析。

主要发现 原型能够在同一图中开展基于轨迹的定性故障分析和定量稳健性测试。论文属于演示轨道，主要证明系统能力，尚未提供充分的人类理解效果证据。

贡献与意义 论文将关系图与精确原始证据链接，是 V1“关联必须可核验”的直接先例；同时也说明系统功能展示不能替代对理解正确率、误判率和无证据结论的受控评估。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：24

名称 Graph of Trace: Visualizing Execution Traces of Scientific Agents

作者 Tianci Gao; Haoxuan Li; Jian He Li; Tianxiang Zhao; Shi Runze; Weiran Wang; Zezhao Wu; Lu Mi 译者

出处 【美国】Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 2026: 297–306, DOI: 10.18653/v1/2026.acl-demo.29 页

文献阅读心得（每篇不少于 150 字）：

研究背景 科学 Agent 可以自动执行环境配置、代码运行、数据分析和结论生成，但展开后的 workflow 难以供领域专家检查，限制了结果复核和后续人机协作。

研究问题

RQ1: 如何实时记录细粒度执行事件并把科学 Agent 工作流组织成可检查的有向图？

RQ2: 结构化轨迹图是否改善领域专家对工作流、失败位置和结果生成过程的理解？

研究方法 系统记录工具调用、代码执行和中间产物，由监视模块构建实时更新的有向图，并提供对话、图和节点细节检查器；作者在复杂科研任务上邀请 AI、神经科学和生物学专家评价。

主要发现 专家认为结构化轨迹提高了工作流理解、感知可解释性和进一步分析交互的可用性。评价以专家反馈为主，尚未通过同证据基线比较理解准确率和无依据判断。

贡献与意义 论文提供与 V1 相近的实时轨迹图和总览—节点细节结构，同时构成重要方法学对照：V1 需要把主观可用性扩展为有 ground truth 的理解和证据判断实验。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：25

名称 AUTOGEN STUDIO: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems

作者 Victor Dibia; Jingya Chen; Gagan Bansal; Suff Syed; Adam Fourney; Erkang Zhu; Chi Wang; Saleema Amershi 译者

出处 【美国】Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2024: 72–79, DOI: 10.18653/v1/2024.emnlp-demo.8 页

文献阅读心得（每篇不少于 150 字）：

研究背景 多 Agent 工作流涉及模型、工具、角色和编排参数，直接通过代码配置和日志排查门槛较高。开发者需要同时查看系统声明结构和具体运行结果。

研究问题

RQ1：声明式规范和无代码界面如何降低多 Agent 工作流构建、调试和评价的复杂度？

RQ2：哪些界面原则能够让模型、工具、Agent 和编排关系被复用并在运行中检查？

研究方法 作者基于 AutoGen 开发 Web 界面和 Python API，以 JSON 声明 Agent 及工作流，提供拖放式结构编辑、交互运行与调试、结果评估和可复用组件库，并通过系统案例说明设计。

主要发现 AutoGen Studio 把工作流结构、组件配置和运行查看统一到一个开发环境中，支持快速原型和非代码式调试。论文主要是系统演示，没有隔离不同呈现方式对理解正确率的影响。

贡献与意义 论文证明“声明结构”和“运行轨迹”可以在同一开发工具中组织，并提供可复用的 Agent 系统表达方式；V1 进一步关心静态声明与本次运行事实之间的证据差异。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：26

名称 AgentTrace: A Structured Logging Framework for Agent System
Observability

作者 Adam AlSayyad; Kelvin Yuxiang Huang; Richik Pal 译者

出处 【美国】arXiv preprint arXiv:2602.10133, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 传统日志偏向操作事件，难以同时表达 Agent 的模型交互、环境动作和上下文状态。静态审计也难以应对由模型推断产生的非确定行为。

研究问题

RQ1: Agent 运行时日志应覆盖哪些观测面，才能支持调试、安全审计和责任追踪？

RQ2: 结构化日志能否在较少侵入 Agent 代码的情况下接入现有遥测基础设施？

研究方法 作者提出 operational、cognitive 和 contextual 三类日志面，设计 schema 化运行时记录机制，并展示其与 OpenTelemetry 等遥测基础设施的集成方式。

主要发现 框架能够持续记录工具和环境动作、模型相关状态以及上下文信息，为实时监控和事后检查提供共同结构。论文以框架设计为主，尚未验证开发者如何理解这些日志。

贡献与意义 论文把上下文状态明确列为与操作和认知并列的观测面，直接支持 V1 聚焦上下文构建与变换；同时提醒研究区分“记录结构完整”与“人能正确理解”两类贡献。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：27

名称 From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents

作者 Yiqi Wang; Jiaqi Zhang; Taotao Cai; Zirui Liu; Qingqiang Sun; Zequn Sun; Zhangkai Wu; Mingkai Zhang; Yanming Zhu 译者

出处 【中国】arXiv preprint arXiv:2606.04990, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 最终答案正确与否不能说明证据如何进入结论、工具调用是否合理、记忆怎样影响后续决策，也不能定位执行失败。相关工作分散在检索归因、工具安全、记忆、可观测和调试领域。

研究问题

RQ1: LLM Agent 中的证据单元、执行单元和溯源关系可以怎样统一分类？

RQ2: 现有方法在表示、粒度、时间、评价和信任功能上有哪些覆盖与缺口？

研究方法 作者开展系统性综述，从 trace 来源、证据与执行单元、provenance 关系、追踪粒度与时机、表示形式和信任功能建立统一分类，并映射数据集、基准和评价指标。

主要发现 现有研究能够追踪检索、工具、记忆和动作的部分来源，但缺少统一 trace schema、语义级溯源、真实执行基准和面向恢复的过程评价，隐私与审计基础设施也不成熟。

贡献与意义 论文直接把证据追踪与执行溯源连接起来，为 V1 定义“关联证据”提供最新综述框架，也支持把评价从最终答案扩展到过程可核验性和证据边界。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：28

名称 LADYBUG: an LLM Agent DeBUGger for Data-Driven Applications

作者 Joel Rorseth; Parke Godfrey; Lukasz Golab; Divesh Srivastava; Jarek Szlichta

译者

出处 【荷兰】Proceedings of the 28th International Conference on Extending

Database Technology, 2025: 1082–1085, DOI: 10.48786/EDBT.2025.94 页

文献阅读心得（每篇不少于 150 字）：

研究背景 数据驱动 Agent 常以多步工作流生成代码、转换数据并汇总结果。某一步的隐蔽错误可能不触发异常，却污染后续结果，用户需要检查中间输入输出并限定重新执行范围。

研究问题

RQ1: 交互式时间线能否支持用户检查 Agent 每一步的输入、输出和代码，并在任意步骤介入？

RQ2: LLM 辅助调试器如何在用户难以发现隐蔽错误时提出候选错误步骤和干预？

研究方法 作者实现 LADYBUG 演示系统，把 Agent 执行表示为按时间排列的步骤，允许用户查看中间状态、修改任意步骤并只重算受影响部分，同时加入基于自我反思的 LLM 辅助调试器。

主要发现 系统能在示例数据应用中定位并修正未直接报错的中间错误，并通过依赖关系减少重算范围。论文是演示短文，没有人类对照实验，也不能把 LLM 提出的原因视为已证实事实。

贡献与意义 论文展示了步骤、依赖和局部重算如何共同支持 Agent 调试。对 V1 而言，其更重要的意义是区分可直接检查的步骤事实、系统计算的依赖关系和 LLM 提出的候选解释。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：29

名称 Open Agent Specification: Enabling Cross-Framework Comparison of AI Agents

作者 Soufiane Amini; Yassine Benajiba; Cesare Bernardis; Paul Cayet; et al. 译者

出处 【美国】Proceedings of the ACM Conference on AI and Agentic Systems, 2026: 404–418, DOI: 10.1145/3786335.3813130 页

文献阅读心得（每篇不少于 150 字）：

研究背景 不同 Agent 框架具有不同的提示处理、工具适配、编排和执行语义。即使 Agent 设计看似相同，运行时也可能产生不同路径和结果，导致跨框架比较缺少共同参照。

研究问题

RQ1: 如何用统一声明表示固定 Agent 结构、数据流和控制流，并在不同运行时中生成可比执行？

RQ2: 相同 Agent 规范在不同框架中是否仍会出现准确率、时延和执行行为差异？

研究方法 作者提出框架无关的 Agent Spec 及标准化评估 Harness，在 LangGraph、CrewAI、AutoGen 和 WayFlow 四种运行时上执行 SimpleQA Verified、 τ^2 -Bench 和 BIRD-SQL 三个基准，并记录统一结构的运行 trace。

主要发现 在共享规范下，不同运行时仍表现出显著的准确率、时延和执行差异，说明框架适配器、提示和编排实现会塑造运行行为，不能由静态规范直接推断。

贡献与意义 论文为结构与运行的对照提供了统一表示和跨框架证据，直接支撑 V1 区分“预设结构”与“本次实际执行”；它也为构造相同任务、不同 Harness 机制的实验材料提供方法。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：30

名称 X-Trace: A Pervasive Network Tracing Framework

作者 Rodrigo Fonseca; George Porter; Randy H. Katz; Scott Shenker; Ion Stoica

译者

出处 【美国】Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation, 2007: 271–284 页

文献阅读心得（每篇不少于 150 字）：

研究背景 复杂网络服务跨越应用、协议和管理域，单层日志不能重建一次用户任务经过的完整路径。即使各组件都产生日志，也缺少将分散事件关联到同一任务的机制。

研究问题

RQ1: 如何在跨层、跨应用执行中传播统一任务标识并重建因果任务树？

RQ2: 该机制在 DNS、Web 和覆盖网络等异构系统中能否提供可用于诊断的端到端视图？

研究方法 X-Trace 在初始任务中注入元数据，并沿协议调用、转发和递归请求传播；各组件记录带共同任务标识的事件，系统据此重建任务树，并在 DNS、三层网站和覆盖网络场景中部署。

主要发现 跨组件任务树能把原本分散的日志连接起来并定位错误数据或异常路径；即使在部分层部署，仍可提供局部诊断价值，但未被记录的分支不能被当作未发生。

贡献与意义 论文奠定了端到端执行溯源的任务树思想，说明可靠关联依赖显式传播的记录机制。它为 V1 界定“直接记录”提供技术基础，也说明信号缺失应标为未知。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：31

名称 Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems

作者 Jonathan Mace; Ryan Roelke; Rodrigo Fonseca 译者

出处 【美国】Proceedings of the 25th ACM Symposium on Operating Systems Principles, 2015: 378–393, DOI: 10.1145/2815400.2815415 页

文献阅读心得（每篇不少于 150 字）：

研究背景 分布式系统中的日志和指标通常预先定义且以单组件为中心，发生未知问题时，开发者难以在运行中提出新的跨组件问题，也难以关联不同位置的事件。

研究问题

RQ1: 能否动态定义监测查询，并在跨机器和跨组件事件之间保留因果关系？

RQ2: 动态因果监测能否以低开销定位软件缺陷、错误配置和性能退化等不同根因？

研究方法 作者结合动态插桩与 happened-before join，使用户可在系统某处定义指标，却按其他组件中的因果先行事件选择、过滤和分组；原型在包含 HDFS、HBase、MapReduce 和 YARN 的 Hadoop 集群中评估。

主要发现 Pivot Tracing 能够跨层查询并定位软件错误、配置问题和慢硬件等多类根因，同时保持较低运行开销。它表明关系可以由明确的因果传播规则计算，而不一定逐条直接记录。

贡献与意义 论文为 V1 的“按规则推导”提供清晰先例：关系有可检查规则和输入证据，但仍不同于原生日志事实。界面应把推导方法、覆盖范围和无法关联的事件一起呈现。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：32

名称 Dapper, a Large-Scale Distributed Systems Tracing Infrastructure

作者 Benjamin H. Sigelman; Luiz André Barroso; Mike Burrows; Pat Stephenson;
Manoj Plakal; Donald Beaver; Saul Jaspán; Chandan Shanbhag 译者

出处 【美国】Google Technical Report dapper-2010-1, 2010 页

文献阅读心得（每篇不少于 150 字）：

研究背景 大规模互联网服务由不同团队、语言和机器上的组件组成。开发者需要理解一次请求跨组件的路径和耗时，但全面插桩会带来开发负担、运行开销与海量数据。

研究问题

RQ1: 如何在低开销、应用透明和大规模持续部署之间取得平衡，记录端到端请求轨迹？

RQ2: 采样式分布追踪是否能支持开发和运维团队理解系统行为并构建多种诊断工具？

研究方法 Dapper 在少量公共库中插桩，以 trace 和 span 记录请求层级及时间信息，通过采样控制开销；论文总结系统在 Google 内部两年多的构建、部署、使用和衍生工具经验。

主要发现 公共库插桩和采样使追踪能够在超大规模生产环境持续运行，并支持性能分析、依赖发现和异常诊断。但采样和未插桩区域意味着轨迹天然不完整。

贡献与意义 论文建立了现代分布式追踪的 trace/span 基础，并揭示观测证据的覆盖边界。

对 V1 而言，界面必须避免把被采样或未插桩造成的空白解释成机制没有运行。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：33

名称 A Controlled Experiment for Program Comprehension through Trace Visualization

作者 Bas Cornelissen; Andy Zaidman; Arie van Deursen 译者

出处 【美国】IEEE Transactions on Software Engineering, 2011, 37(3): 341–355, DOI: 10.1109/TSE.2010.47 页

文献阅读心得（每篇不少于 150 字）：

研究背景 执行轨迹可视化常被用于程序理解，但早期工具多依赖案例演示，缺少把可视化与常规开发环境直接比较的量化证据。

研究问题

RQ1：大型执行轨迹可视化是否能提高典型程序理解任务的正确率？

RQ2：在提供相同目标系统的情况下，可视化是否能减少完成理解任务所需的时间？

研究方法 作者设计 8 项面向代表性软件系统的程序理解任务，以 Eclipse 作为对照条件，以 Eclipse 加 Extravis 执行轨迹可视化作为实验条件，比较完成时间和答案正确性。

主要发现 使用轨迹可视化的实验组完成任务所需时间降低 22%，正确性提高 43%，两项差异均达到统计显著。研究证明结构化轨迹的价值可以用客观理解任务而非满意度评价。

贡献与意义 论文为 V1 的三条件受控实验提供直接方法学依据：应保持任务和证据可比，测量理解正确率与时间，并把“可视化好看”与“实际理解更准确”区分开。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：34

名称 Debugging Strategies and Tactics in a Multi-Representation Software Environment

作者 Pablo Romero; Benedict du Boulay; Richard Cox; Rudi Lutz; Sallyann Bryant
译者

出处 【英国】International Journal of Human-Computer Studies, 2007,
65(12): 992–1009, DOI: 10.1016/j.ijhcs.2007.07.005 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试环境同时提供代码、图形和运行输出时，用户不仅要理解每种表示，还要在表示之间切换和建立对应关系。表示越多并不自动带来更好表现。

研究问题

RQ1: 程序员如何把高层调试策略与低层界面操作结合，并在多种表示之间组织搜索？

RQ2: 编程经验、图形表示知识和呈现形式如何影响调试准确率及策略适应？

研究方法 研究观察不同经验水平的程序员在多表示软件环境中完成调试任务，结合任务表现和交互过程分析他们查看代码、图形和输出的时间、切换方式及策略变化。

主要发现 成功者更系统地把代码与输出对应起来，并能随呈现条件调整策略；经验较少者更频繁地在表示间切换，形成重复且低效的浏览模式。掌握外部表示规则与编程经验同样重要。

贡献与意义 论文说明 V1 的结构图、轨迹和上下文视图必须有明确对应关系，不能只增加表示数量。评价还应观察跨视图跳转、回退和关系核验，而不只记录最终答案。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：35

名称 Are Automated Debugging Techniques Actually Helping Programmers?

作者 Chris Parnin; Alessandro Orso 译者

出处 【加拿大】Proceedings of the 20th International Symposium on Software Testing and Analysis, 2011: 199–209, DOI: 10.1145/2001420.2001445 页

文献阅读心得（每篇不少于 150 字）：

研究背景 自动故障定位常以错误语句排名为目标，并假设程序员只要看到正确位置就能理解和修复问题。然而真实调试还包含依赖导航、状态检查、重跑和根因理解。

研究问题

RQ1: 现有自动调试研究关于程序员如何使用故障排名的假设是否符合实际调试行为？

RQ2: 调试技术应如何评价，才能反映其是否真正帮助开发者理解根因并完成修复？

研究方法 作者分析自动调试和故障定位方法的常见假设，并通过面向开发者的调试任务考察排名信息如何被检查、解释和用于后续导航，进而反思只以错误位置排名评价工具的做法。

主要发现 程序员不会机械地逐项检查排名，也不能仅凭孤立语句理解根因；即使正确语句排名靠前，用户仍需控制流、数据流、状态和上下文证据才能判断。

贡献与意义 论文确立了“自动定位准确”不等于“人能够理解并行动”的方法学原则。V1 因此应直接测人的事件—机制映射和证据定位，而不能用自动算法指标替代人侧实验。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：36

名称 Characterizing Provenance in Visualization and Data Analysis: An
Organizational Framework of Provenance Types and Purposes

作者 Eric D. Ragan; Alex Endert; Jibonananda Sanyal; Jian Chen 译者

出处 【美国】IEEE Transactions on Visualization and Computer Graphics,
2016, 22(1): 31–40, DOI: 10.1109/TVCG.2015.2467551 页

文献阅读心得（每篇不少于 150 字）：

研究背景 可视分析中的 provenance 既可指数据来源、系统操作历史，也可指分析者的推理和洞见。概念混用会导致系统记录了大量历史，却不能回答这些记录服务什么任务。

研究问题

RQ1: 可视化与数据分析中的 provenance 可按哪些信息类型和使用目的组织？

RQ2: 不同 provenance 类型应如何影响捕获方式、界面设计和评价方法选择？

研究方法 作者综合多个领域的 provenance 系统和研究，对记录对象、语义层次、捕获方式及使用目的进行交叉整理，提出组织框架，并讨论类型、目的和评估之间的关系。

主要发现 provenance 不仅是操作日志，还包括数据、可视状态、分析步骤、推理和洞见等不同层次；回溯、复现、协作、审计和理解等目的需要不同信息，不能由单一历史记录满足。

贡献与意义 论文为 V1 区分运行事实、界面推导和人的解释提供概念工具，也说明证据状态必须与具体关系和使用目的绑定，而不能只给整条轨迹一个统一可信度。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：37

名称 Analytic Provenance for Sensemaking: A Research Agenda

作者 Kai Xu; Simon Attfield; T. J. Jankun-Kelly; Ashley Wheat; Phong H. Nguyen;
Nallini Selvaraj 译者

出处 【美国】IEEE Computer Graphics and Applications, 2015, 35(3): 56–
64, DOI: 10.1109/MCG.2015.50 页

文献阅读心得（每篇不少于 150 字）：

研究背景 意义建构包含信息搜索、假设生成、证据比较和框架修正。若系统只保存点击而不保存分析步骤及其关系，就难以支持反思、恢复中断或追踪不确定性。

研究问题

RQ1：分析过程中的数据、操作、子任务、假设和推理应以怎样的层级被捕获和呈现？

RQ2：analytic provenance 如何支持意义建构、协作以及不确定性和信任管理？

研究方法 论文汇总 IEEE VIS 相关研讨会和既有研究，围绕 provenance 的捕获、可视化和利用提出研究议程，并用分层模型讨论从低层交互到高层任务语义的推断。

主要发现 低层事件容易自动记录，高层任务和意图难以可靠推断；provenance 可支持过程反思、探索引导和质量追踪，但推断、隐私、规模化与不确定性传播仍是开放问题。

贡献与意义 论文直接连接 provenance 与 Data-Frame 意义建构，支持 V1 把“证据从哪里来、怎样进入结论”作为界面主结构，并要求对自动推断的高层语义显示不确定性。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：38

名称 Capturing and Supporting the Analysis Process

作者 Nazanin Kadivar; Victor Chen; Dustin Dunsmuir; Eric Lee; Cheryl Qian; John Dill; Christopher Shaw; Robert Woodbury 译者

出处 【美国】Proceedings of the IEEE Symposium on Visual Analytics Science and Technology, 2009: 131–138, DOI: 10.1109/VAST.2009.5333020 页

文献阅读心得（每篇不少于 150 字）：

研究背景 分析者会反复调整参数、返回旧状态并探索替代路径，但多数可视分析工具只保存当前结果，不能让人查看、恢复或理解分析过程本身。

研究问题

RQ1：如何捕获并可视化分析者的交互历史、分支和结果依赖？

RQ2：可编辑、可回放的分析历史能否支持过程理解、替代路径探索和分析模式复用？

研究方法 作者开发 CzSaw，把用户交互翻译为脚本，生成可编辑的历史视图和结果依赖图，并通过文档集合分析场景展示状态恢复、分支探索和分析流程复用。

主要发现 历史视图使用户能够查看分析进展和替代路径，依赖图则揭示每一步与后续结果之间的逻辑关系；脚本化记录还支持修改和重新执行分析步骤。

贡献与意义 论文提供了“时间历史+依赖关系”双重表示的经典先例，与 V1 的轨迹和机制并列结构高度对应；它也提示两类视图应联动而非各自展示重复信息。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：39

名称 The Role of Uncertainty, Awareness, and Trust in Visual Analytics

作者 Dominik Sacha; Hansi Senaratne; Bum Chul Kwon; Geoffrey Ellis; Daniel A. Keim 译者

出处 【美国】IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 240–249, DOI: 10.1109/TVCG.2015.2467591 页

文献阅读心得（每篇不少于 150 字）：

研究背景 可视分析系统从数据、模型到界面和人的解释都会引入或继承不确定性。若使用者看不到不确定性如何传播，就可能对结果形成过强或过弱的信任。

研究问题

RQ1：不确定性如何在可视分析的数据处理、模型、可视化和人类推理之间传播？

RQ2：不确定性意识如何影响证据搜索、验证循环和信任形成？

研究方法 作者整合可视分析、认知和信任研究，提出包含知识生成、探索、验证、不确定性传播与信任构建的概念模型，并通过分析场景说明各环节关系。

主要发现 不确定性会在系统组件中继承和累积，使用者的意识决定其是否继续寻找证据、质疑结果或采取行动；信任应建立在可检查证据和验证循环上，而非结果外观。

贡献与意义 论文为 V1 的证据状态提供理论支撑：界面不仅要显示关系，还要让使用者意识到关系的证据强度与未知范围，从而减少由完整结构图诱发的过度确定。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：40

名称 Why Authors Don't Visualize Uncertainty

作者 Jessica Hullman 译者

出处 【美国】IEEE Transactions on Visualization and Computer Graphics,
2020, 26(1): 130–139, DOI: 10.1109/TVCG.2019.2934287 页

文献阅读心得（每篇不少于 150 字）：

研究背景 尽管不确定性表达方法很多，公开可视化仍经常给出确定性的单一叙事。问题不仅是缺少图形技术，还包括作者对受众、说服力和信息复杂度的假设。

研究问题

RQ1: 经常制作可视化的作者为何承认不确定性重要，却仍选择不直接呈现？

RQ2: 哪些工作实践、态度和传播目标维持了不确定性省略的规范？

研究方法 研究调查 90 名经常为他人制作可视化的作者，并访谈 13 名有影响力的可视化设计者；作者据此归纳实践障碍和信念矛盾，提出不确定性省略的修辞模型。

主要发现 作者普遍认可不确定性的价值，但担心它削弱清晰信息、增加受众负担或破坏叙事。现实中不确定性因此经常被省略，完整而确定的图形外观会掩盖推断自由度和证据边界。

贡献与意义 论文解释了为什么“证据状态应显示”并不会自然转化为界面实践。V1 需要验证三类状态怎样呈现才既不造成认知负担，也不让结构图以确定外观掩盖未知

二、国内外研究综述

1 国内外研究综述

大语言模型 (Large Language Model, LLM) Agent 已由单轮问答扩展为能够调用工具、访问环境、维护记忆并持续执行任务的复合系统。此类系统的行为不仅由基础模型决定, 还受到提示组织、上下文装配、工具接口、控制循环、状态管理、错误处理和验证规则等运行基础设施的共同塑造。本文将这些围绕模型、负责把一次任务组织成可执行过程的基础设施统称为 Agent Harness。对开发者而言, 真正困难的问题已不只是判断任务是否成功, 而是解释某一运行现象如何产生、哪些轨迹证据与之相关, 以及这些证据如何连接到具体的 Harness 机制。围绕这一问题, 现有研究主要分布在 LLM 产品评价、程序调试与意义建构、Agent 可观测与交互调试、分布式追踪与分析溯源、不确定性表达及人因评价等领域。下面依次梳理这些研究, 并据此界定本研究的问题边界。

1.1 Agent Harness 与运行理解的概念边界

1.1.1 从模型评价到系统运行评价

早期 LLM 研究通常把模型能力作为主要评价对象, 以准确率、胜率或基准任务得分衡量效果。然而, 真实产品中的输出是模型、提示、检索、界面、业务规则与人员协作共同作用的结果。van der Maden 等对 19 名 LLM 产品实践者的访谈表明, 17 名受访者都经历过“结果—可行动性鸿沟”: 现有评价能够揭示产品表现, 却难以指出下一步应修改哪个系统部分[1]。这一发现说明, 当评价对象从孤立模型转向完整 Agent 系统时, 最终分数无法替代运行机制层面的解释。Khanna 等提出的 AAR/AI 也表明, 相较于总体性能指标, 把问题定位到具体决策与内部预测能够帮助使用者发现更多 AI 故障; 在具有已知真值的任务中, 该方法提高了问题召回率和识别故障的可能性, 但精确率仍然有限[4]。由此可见, 运行理解既需要比结

果指标更细粒度的证据，也必须防止把局部异常过早解释为已确定的根因。

Harness 概念进一步揭示了系统评价为何不能等同于模型评价。Chen 等把 Harness 划分为执行环境与沙箱、工具接口、上下文与记忆、生命周期与编排、可观测、验证评估、治理安全等层，并指出失败轨迹中的决定性证据往往散落在多个层次[20]。Lin 等则把系统提示、工具、中间件、技能、子 Agent、长期记忆和配置视为可编辑 Harness 组件，强调高性能 Agent 依赖的不只是提示词，而是组件之间可被追踪、比较和回滚的工程结构[21]。Amini 等在四种运行时上执行相同的 Agent 声明，仍观察到准确率、延迟和执行行为的明显差异，说明静态规格相同并不保证运行语义相同[29]。因此，本研究中的“Harness 运行理解”并非阅读配置文件，也不是概括 Agent 的自然语言理由，而是把实际执行过程与塑造该过程的运行机制联系起来。

1.1.2 运行轨迹、机制关联与故障归因

Agent 运行轨迹通常包含模型请求与响应、工具调用与结果、文件或环境变化、Agent 间消息、重试与终止等事件。轨迹记录回答“发生了什么”，Harness 机制回答“系统以什么规则组织了这些事件”，两者之间的关联才可能支持“为什么出现该现象”的解释。Zhang 等把多 Agent 故障归因分解为责任 Agent 和责任步骤两个维度，在 127 个系统、184 个归因任务上的结果显示，即使先进方法对责任 Agent 的识别准确率达到 53.5%，对具体步骤的准确率也只有 14.2%[3]。这说明长轨迹中的“谁”和“何时”本身就难以可靠确定，更不能把自动归因结果直接当作事实呈现。

同时，运行理解与自动修复应当区分。HarnessFix 把原始轨迹与 Harness 工件编译为中间表示，通过数据流、控制流和实现锚点把失败步骤映射到责任层，再生成并验证修补方案[20]；AHE 则通过组件可观测、经验可观测和决策可观测构造 Harness 自动演化闭环[21]。两类工

作证明了轨迹—机制关联在技术上可被计算，也显示这种关联能够支持系统改进。但它们主要以任务性能、修复收益和跨模型迁移为目标，尚未回答开发者如何检查关联证据、如何区分直接记录与算法推断，以及界面呈现是否会造成新的误解。因此，本文所说的运行理解以人的证据判断和故障定位为中心，不以自动生成修复方案为首要目标。

1.1.3 上下文构造作为关键研究切口

在 Agent Harness 的诸多机制中，上下文构造具有连接模型、工具和历史状态的枢纽作用。一次请求实际接收哪些系统指令、历史消息、工具结果与检索材料，哪些内容被保留、替换、裁剪或摘要，直接改变模型下一步可见的信息。AgentTrace 把上下文事件与认知事件、操作事件并列为三类结构化日志，说明仅记录工具调用无法重建 Agent 当时的输入条件[26]。AgentSight 从系统边界同时观测模型侧语义意图和内核侧行为后发现，高层意图与低层效果之间存在明显语义鸿沟[19]。Open Agent Specification 的跨框架实验进一步表明，即使共享声明结构，不同运行时对提示和工具的处理仍会造成行为差异[29]。这些研究共同支持以“信息在何处进入请求、如何被转换、是否仍可被后续步骤看见”为核心分析对象。相较于覆盖所有 Harness 层，这一切口既与轨迹证据直接相连，也便于构造具有客观真值的实验案例。

1.2 开发者调试与意义建构的认知基础

1.2.1 从异常现象到 why 与 why-not 问题

传统调试研究指出，开发者通常先观察到异常输出，再逐步提出关于原因的假设。Katz 和 Anderson 将“定位错误”与“判断、修复错误”区分开来，发现即便程序员有能力修复已经指出的错误行，也未必能从行为表现中有效找到该位置[15]。这一区分对 Agent 运行理解尤为重要：用户能够看懂某条工具报错，并不意味着能够发现导致该报错的上下文裁剪或控制分支。

Ko 和 Myers 围绕 why 与 why-not 问题形成的一系列研究,把调试入口从“先猜代码位置”改为“直接询问可见行为为什么发生或为什么没有发生”。Whyline 界面允许用户针对程序输出提出问题,再由系统生成相关运行时因果链,早期研究显示其能够显著缩短问题定位时间[6]。后续 Java Whyline 扩展到真实 Java 程序,在受控实验中提高了成功完成调试任务的比例,并减少了定位时间[7]。在此基础上,“Debugging Reinvented”进一步论证:如果工具能围绕开发者实际拥有的疑问组织证据,就能减少断点与打印语句造成的无关搜索[5]。这些研究的核心价值不在于复用某一种传统调试器控件,而在于把界面组织单位从原始事件转为用户可提出的问题,并保留问题到证据之间可检查的联系。

Agent 系统已经开始吸收这一思路。AGDebugger 面向多 Agent 消息轨迹提供暂停、重置和修改消息等交互,使开发者能够在错误传播前探索替代路径;形成性访谈和 14 人实验表明,交互调试有助于定位问题,但参与者在需要正确选择干预位置和方式时仍频繁失败[2]。AgentStepper 把断点、单步执行、中间状态检查和提示或工具调用编辑引入软件开发 Agent。其 12 人研究中,参与者识别 Agent 实现缺陷的成功率由 17%提高到 60%,挫败感由 5.4 降至 2.4,但一般轨迹解释成绩的提升并不明显[18]。这意味着“能操控轨迹”与“能形成正确解释”不是同一件事,交互功能必须与证据结构和机制关联共同设计。

1.2.2 信息觅食、层级搜索与跨表示协调

调试过程也可被理解为在信息空间中寻找具有价值的线索。Lawrance 等以信息觅食理论重新分析专业程序员的调试行为,发现开发者根据界面线索形成“信息气味”,并在文件、方法和调用关系组成的“信息拓扑”中决定继续追踪还是转换路径[8]。Ko 等对软件维护任务的观察显示,参与者平均约 35%的有效时间用于导航、关联和收集信息,说明工具切换和线索组织本身就是理解成本的重要来源[14]。Rasmussen 和 Jensen 对真实电子故障排查的研究则表

明，熟练人员会在系统、功能单元和部件之间进行层级搜索，并在信息收益、测量成本与认知负荷之间折衷[9]。这些结果提示，Agent 调试界面不应要求用户从头顺序阅读完整日志，而应提供从异常症状下钻到局部事件、再上溯到机制的可导航拓扑。

但增加视图不一定带来更好理解。Romero 等研究同时呈现代码、输出与图形表示的调试环境，发现成功表现不仅依赖编程经验，也依赖用户理解外部表示并协调不同表示的能力；较有经验者能够根据任务调整策略，经验较少者却可能承受额外切换成本[34]。Cornelissen 等对执行轨迹可视化的受控实验显示，在八项程序理解任务中，轨迹可视化组用时减少 22%，正确率提高 43%[33]，证明结构化轨迹具有潜在价值；然而该结果并不意味着任意可视化都有效，因为实验中的收益来自任务、视图和交互之间的具体匹配。对本研究而言，时间轴、关系图、机制代码与证据详情必须通过稳定标识和明确选中状态相互对应，避免用户在多个视图之间重新寻找同一事件。

1.2.3 Data – Frame 意义建构与困惑消解

运行理解不是一次性读取事实，而是持续构造和修正解释框架。Klein 等对不同 sensemaking 传统的梳理指出，复杂工作中的理解包含联系信息、解释意外、维持连贯叙事并支持下一步行动等活动[10]。其宏观认知模型强调，既有框架会引导人关注特定数据，新的数据又可能强化、质疑或替换该框架[11]。Data – Frame 理论进一步把这一过程描述为框架与数据之间的双向循环：人利用框架解释线索，当异常无法纳入当前框架时，便进行质疑、比较和重构[12]。Agent 运行中的一次异常工具结果、缺失消息或意外终止，正是触发框架更新的关键线索。

Pirolli 和 Card 通过认知任务分析把意义建构划分为信息觅食循环与意义建构循环：前者负责搜索、筛选和组织材料，后者负责建立证据结构、生成假设并形成结论[17]。这一模型说

明，支持运行理解的界面既要降低查找证据的成本，也要允许用户保留和比较暂时解释。

Kadivar 等的 CzSaw 系统通过记录分析历史、依赖关系、分支和回放，使分析者能够返回旧状态、探索替代路径并复用已有过程[38]。Xu 等进一步指出，低层点击容易自动捕获，高层任务、意图和推理却难以准确推断；分析溯源若把自动推断的高层语义伪装成原始记录，反而会破坏反思与协作[37]。

Starr 和 Storey 从 27 名开发者访谈中总结了故障排查中的困惑、试探、观察和恢复过程，指出困惑会占用注意与工作记忆，并随时间转化为疲劳[16]。这为 Agent 界面的设计提供了重要人因边界：系统不应以更多字段和图形制造“可观测即理解”的假象，而应围绕当前问题逐步揭示关联信息；同时，应保留用户回退、比较和修正判断的空间。换言之，理想界面不是一次给出完整答案，而是帮助开发者在证据约束下形成、检查并更新自己的 Data - Frame。

1.3 Agent 轨迹可观测与交互调试研究进展

1.3.1 从结构化日志到多粒度轨迹

Agent 可观测研究首先解决“记录什么”。AgentTrace 提出操作、认知和上下文三个日志平面，并通过统一的 trace id 与 span id 组织模型交互、环境动作和上下文变化[26]。该框架把运行日志从运维附属物提升为安全、复现和问责的基础，但目前主要贡献是日志模式，尚未验证不同呈现方式对开发者理解的影响。AgentSight 采用 eBPF 在不修改 Agent 代码的情况下捕捉模型通信和系统事件，通过边界追踪关联高层意图与低层效果，开销低于 3%[19]。它拓展了可观测的覆盖面，也带来新的证据问题：系统调用是直接观测到的，语义意图及二者关系可能包含模型分析或规则推断，界面必须让用户识别这种差异。

传统分布式追踪为 Agent 轨迹组织提供了成熟概念。X-Trace 通过显式传播任务标识，把跨

应用、协议和管理域的事件重建为任务树[30]；Dapper 以 trace 与 span 表达一次请求的分布式路径，并以通用库插桩和采样在覆盖率、透明度与运行开销之间折衷[32]。这些系统说明，可靠关联依赖稳定标识和因果上下文传播，而不是事后仅凭时间邻近猜测。但采样、未插桩组件或标识传播中断都会造成空白，因此“轨迹中没有记录”不能直接推出“事件没有发生”。Pivot Tracing 进一步引入 happened-before join，使用户能够在运行时提出跨组件查询并依据先发生关系连接事件[31]。它展示了规则推导关联的价值，也提示推导结果应附带所用规则与输入证据。

1.3.2 面向人的观察、调试与干预工具

现有 Agent 工具大致形成三类界面。第一类以时间和层级结构帮助用户浏览长轨迹。AgentLens 从原始事件构建行为结构，以层级时间视图展示多 Agent 行为演化，并允许从总体模式下钻到具体行为及候选原因[22]。AgentGUI 同时呈现任务、轨迹、产物和成本，并提供人工与自动纠偏；其实验报告轨迹元素识别速度提高 38%，识别准确率由 80% 提升至 93%[13]。这些工具证明了概览—细节协同的价值，但候选原因或自动管理建议仍可能包含推断，若缺少证据强度标记，用户容易把视觉邻近或系统推荐理解为确定因果。

长轨迹界面还面临“压缩与保真”的矛盾。概览必须把成百上千个事件归纳为可浏览单元，否则用户仍会陷入线性阅读；但摘要又可能删除决定性参数、异常返回值或消息边界。

AgentLens 通过层级聚合压缩事件，Graph of Trace 通过有向图突出阶段和依赖，AgentGUI 通过任务和产物视图减少终端噪声[13][22][24]。这些方法都在降低搜索成本，却没有彻底解决摘要与原始记录的对应问题。对运行理解而言，任何被压缩的节点都应能展开到原始事件，摘要文本应被视为一种派生表示，而不是原始证据本身；若摘要由 LLM 生成，还应说明生成范围和遗漏风险。该要求与 Whyline 的设计原则一致：界面可以替用户缩小搜索空

间，但必须允许用户检查系统据以回答问题的实际运行证据[5][7]。

第二类以结构图重建任务与依赖关系。AgentGraph 把 Agent、任务、工具、输入输出和人员转为知识图谱，每个图元素都链接到精确轨迹片段，因而具有较好的可核验性[23]。Graph of Trace 把科学 Agent 的执行事件组织成持续更新的有向图，领域专家认为结构化轨迹有助于理解 workflow、检查中间步骤和后续协作[24]。两者均说明图结构适合表达长链依赖，但现有评价多是系统展示、使用情景或主观反馈，尚未充分测量用户能否准确区分已记录关系、算法重建关系与未知关系。

第三类将观察与干预结合。AutoGen Studio 以声明式 JSON 描述 Agent 与 workflow，通过拖拽界面、运行游乐场和消息查看支持多 Agent 原型构建与调试[25]；LADYBUG 允许用户检查每一步输入输出、在任意步骤干预并只重算受影响部分，同时利用 LLM 提出候选错误与修正[28]；AGDebugger 与 AgentStepper 则进一步支持暂停、回退、单步和现场编辑[2][18]。这类工具降低了试验替代路径的成本，却也把“观察”“判断”和“改变系统”紧密耦合。若用户尚未确认原因时直接改写轨迹，新的结果可能被误当作对原假设的验证。因此，面向运行理解的界面应先支持证据定位与机制核对，再把干预作为可选的后续活动。

表 1 代表性 Agent 运行可观测与交互调试系统比较

代表工作	主要表示	核心交互	轨迹—机制关联	证据状态呈现	人因评价特点
AGDebugger[2]	多 Agent 消息轨迹	暂停、重置、编辑消息	主要关联消息与 Agent 角色	未显式区分	形成性访谈与 14 人实验，关注定位和干预
AgentGUI[13]	任务、轨迹、产物与成本视图	观察、人工/自动纠偏	可定位轨迹元素，机制映射有限	未系统区分	测量时间与正确率
AgentStepper[18]	结构化会话与代码变化	断点、单步、编辑、检查状态	关联 Agent 动作与实现执行	未显式区分	12 人对照研究，测量缺陷识别和负荷
AgentLens[22]	层级时间视图	概览、筛选、下钻、原因追踪	部分原因由算法挖掘	推断边界不突出	使用情景与用户研究
AgentGraph[23]	交互知识图谱	图浏览、回到原始轨迹	图元素链接精确 trace span	可核验，但状态分类有限	系统展示为主

代表工作	主要表示	核心交互	轨迹—机制关联	证据状态呈现	人因评价特点
Graph of Trace[24]	实时有向工作流图	过程检查与阶段回顾	依赖由监测与解析模块重建	未独立编码	领域专家主观评价
HarnessFix[20]	Harness-aware 中间表示	自动归因与修复	数据流、控制流、实现锚点	面向机器诊断	以基准性能和修复效果为主

1.3.3 自动归因、修复与人的理解之间的断层

Agent 故障诊断研究正在从“显示轨迹”走向“解释轨迹”。Who&When 尝试自动判断责任 Agent 与步骤，但步骤级准确率仍低，表明归因模型本身需要被审查[3]。HarnessFix 将轨迹片段映射到 Harness 层与实现工件，并在四类基准上通过针对性修复获得性能提升[20]；AHE 把每次 Harness 编辑与预期结果写成可证伪契约，并用下一轮任务结果验证[21]。这些方法的共同贡献是：它们不再把失败轨迹仅视为负向奖励，而把轨迹视为定位机制问题的结构化证据。

不过，自动系统的正确率或修复收益不能直接证明开发者获得了正确理解。Parnin 和 Orso 对自动故障定位工具的研究指出，算法把错误语句排在前列，并不意味着程序员能够理解根因；真实调试还需要沿数据和控制依赖导航、检查状态并通过重跑检验假设[35]。这一批评同样适用于 Agent 归因：一个“责任步骤”标签若没有展示其数据来源、关联规则与缺失信息，可能提高操作速度，却也可能诱发过度确定。由此形成当前研究的关键断层：技术系统已能产生丰富的关联和候选解释，人机交互研究却较少验证这些解释如何被开发者理解、误解和校准。

从可行动性角度看，人机界面与自动修复系统还具有不同的输出要求。自动修复可以在大量候选中搜索，并以回归测试筛选有效补丁；开发者界面则必须让人理解当前判断为何成立，以及不成立时应从哪里继续调查。AHE 把一次编辑前的预测与下一轮结果绑定，强调“修改—预期—验证”的可审计链[21]；HarnessFix 使用实现锚点限制修复范围，避免从局部现象跳

到宽泛改动[20]。若把这种思想转向人机交互，界面应支持开发者把每项判断写成可验证主张，并指出它由哪些记录和规则支持、哪些替代解释尚未排除。这样，运行理解的产物就不只是一个根因标签，而是能够被同事复核、被后续运行证伪或强化的证据化解释。

1.4 轨迹—机制关联的证据基础

1.4.1 provenance 的对象与用途

Provenance 研究为“从哪里来、经过什么过程、为何形成当前结果”提供了系统概念。Ragan 等梳理可视化与数据分析中的 provenance，将其区分为数据、可视化、交互、洞见和理由等类型，并强调记录类型必须与使用目的相匹配[36]。如果目标是复现运行，保存参数、输入和执行环境可能已经足够；如果目标是解释开发者为何形成某一判断，还需要假设、注释和证据选择过程。Wang 等针对 LLM Agent 进一步提出统一的执行溯源框架，把检索证据、工具结果、记忆项、环境观察、中间声明、动作与最终答案组织为带类型的关系，并指出现有工作仍缺少统一模式、语义级 provenance 和面向恢复过程的评价[27]。

分析溯源还揭示了语义层级问题。Xu 等提出从低层事件、分析动作、子任务到总体任务的层次模型，指出语义越高，越难仅靠系统日志可靠推断[37]。CzSaw 通过自动记录交互历史并允许用户编辑、分支和重放，在机器记录与人的解释之间建立互补关系[38]。对 Agent Harness 而言，工具返回文本、请求长度、消息删除等可以直接记录；“这条结果因上下文裁剪而未被模型使用”可能需要结合数据流规则推导；“模型为何忽略仍然可见的信息”则可能超出当前证据能力。界面若把三者以同样样式呈现，就会掩盖语义跃迁和推断风险。

1.4.2 记录、推导与未知的证据状态

基于分布式追踪、Agent 可观测和 provenance 研究，可以把面向运行理解的证据划分为三类状态。第一类是“实际记录到”，指系统在运行时直接捕获且能够回到原始事件的事实，例如

某次模型请求的消息列表、某个工具调用参数、某条文件写入事件。第二类是“按规则推导”，指依据显式规则把多个已记录事实连接起来，例如通过 `trace id` 重建父子 `span`，通过 `happened-before` 关系确定先后依赖，或通过 `Harness` 代码中的构造函数把输入字段映射到请求片段[30][31][32]。第三类是“目前无法知道”，指由于未插桩、采样、状态未持久化或模型内部过程不可观测，现有证据不能支持肯定结论。

这种分类并非简单的可信度高、中、低。直接记录也可能不完整或来自错误时钟；规则推导可以高度可靠，但必须暴露规则和前提；未知则是一种必要结论，而不是系统失败。Sacha 等指出，不确定性会从数据、模型、可视化到人的解释逐层传播，用户对系统的信任取决于是否意识到这些不确定性[39]。Hullman 对 90 名可视化作者的问卷和 13 名设计者的访谈发现，作者常因担心复杂度、说服力下降和读者认知负担而省略不确定性，尽管他们同时承认不确定性有价值[40]。这揭示了一个重要设计矛盾：证据状态不能被隐藏，但也不宜用密集概率或警告淹没主要任务。更合适的方式是以稳定、可解释的状态标识呈现，并允许用户查看状态依据。

证据状态的呈现还应避免把“未知”处理成视觉上的次要信息。调试活动具有强烈的闭合倾向：当界面给出一条流畅的候选原因或醒目的高亮路径时，用户可能停止寻找反证。Data - Frame 理论认为，异常线索是触发框架重构的重要条件[12]；若未知信息被折叠，用户将难以发现当前框架不能解释的部分。因而，状态设计至少需要完成三项功能：让用户一眼区分事实与推导；在推导处提供可展开的规则、输入和反例条件；在未知处说明缺失信号及可能的补证方式。对于无法通过本次运行补证的问题，界面也应允许用户保留“尚不能判断”，而不是强迫选择单一原因。这里的目标并非削弱信任，而是把信任从对系统权威的笼统接受转化为对具体证据链的校准。

表 2 Agent Harness 运行理解中的证据状态

证据状态	判定依据	典型例子	界面需要回答的问题	主要风险
实际记录到	有原始事件、时间戳与稳定标识，可返回源记录	请求中实际包含的消息；工具真实返回值；文件写入	记录位于何处，是否完整，属于哪次运行	把“记录到结果”误解为“已证明原因”
按规则推导	由已记录事实和公开规则计算，能够说明输入与规则	由父子 span 还原调用链；由代码锚点推得上下文来源	使用了什么规则，前提是否满足，能否复算	把算法推断误当作直接观察
目前无法知道	必需信号缺失或超出系统可观测边界	未保存的中间上下文；模型内部为何选择某动作	缺少什么，是否能通过补充插桩确认	用时间邻近、相关性或语言流畅性填补空白

1.4.3 从可追溯关系到可辩护结论

轨迹—机制关联的价值不只在生成更多连线，而在于缩短从现象到可辩护结论的路径。X-Trace 和 Dapper 说明，跨组件关联若要稳定，需要在执行过程中传播上下文[30][32]；Pivot Tracing 说明，事前没有记录的指标可通过动态插桩和关系查询补充，但查询结果仍依赖明确的因果顺序[31]；AgentGraph 通过让图元素返回精确轨迹片段，提高了关系的可核查性[23]；HarnessFix 再向前一步，把运行步骤连接到 Harness 工件和修复算子[20]。这些工作共同形成三种关联证据：信息从哪里流入和流向何处的数据流证据，哪个分支、重试或终止规则被触发的控制流证据，以及哪个配置项、函数或文件实现该机制的实现锚点。

本研究因此不把关系图中的“边”视为同质对象。界面需要说明每条边表达的是观测到的数据传递、规则推导的控制依赖，还是面向实现的静态映射；同时还要允许用户从边回到相关轨迹与代码位置。只有这样，用户才能判断结论的适用范围。例如，“工具结果出现在日志中”只能证明结果产生过；若快照显示该结果未进入下一次模型输入，才可进一步判断上下文传递发生中断；若快照缺失，则应保留未知状态，而不是仅凭后续输出推断结果被丢弃。

1.5 现有研究的评价方法

1.5.1 从系统展示到受控理解任务

Agent 可观测研究的评价大致分为三类。第一类是系统展示和使用情景，常用于验证能否接入真实框架并呈现复杂轨迹，例如 AutoGen Studio、AgentGraph、LADYBUG 和 AgentTrace[23][25][26][28]。这类研究能够说明功能可实现，却难以判断开发者是否因此形成更准确的机制解释。第二类是专家访谈和主观反馈，例如 Graph of Trace 邀请不同学科专家评价可理解性和可用性[24]，AgentLens 结合使用情景与用户研究考察行为分析[22]。它们适合发现工作流和界面问题，但主观“觉得清楚”可能与客观正确率不一致。

第三类是具有对照条件和客观任务的实验。Whyline 系列使用预设故障与可判断任务完成标准比较调试表现[5][6][7]；轨迹可视化实验比较常规 IDE 与增加可视化后的完成时间和正确率[33]；AAR/AI 以十个已知故障作为真值，分别测量召回率与精确率[4]；AgentGUI 和 AgentStepper 分别测量轨迹元素识别、缺陷发现、时间与工作负荷[13][18]。这类设计更接近本研究需要，因为“运行理解”必须通过参与者回答与客观运行机制之间的吻合程度来评价，而不能只依赖可用性量表。

1.5.2 指标选择与实验条件隔离

现有研究提示，运行理解至少包含四类可测结果。第一是理解正确性，包括对事件顺序、数据来源、机制作用和故障位置的判断；第二是证据定位能力，包括找到关键轨迹片段、机制工件和二者关系的时间与成功率；第三是结论校准，包括无依据断言、把推导说成事实、在证据不足时拒绝判断等行为；第四是过程与负担，包括完成时间、导航路径、视图切换、NASA-TLX 及主观信任。只报告任务性能会遗漏人的代价，只报告满意度又无法证明理解正确。

此外,实验需要隔离“增加了什么信息”和“如何组织相同信息”的效应。Parnin 和 Orso 指出,自动调试研究常把算法排名质量等同于对程序员的帮助[35]; Romero 等显示,多种表示的加入可能同时改变信息量和协调负担[34]; 不确定性研究则说明,额外标记既可能校准信任,也可能增加认知成本[39][40]。因此,如果实验组同时获得更多底层日志、更强关联算法和更丰富界面,就无法判断收益来自哪一部分。较稳妥的设计是使用共享数据底座,依次比较原始日志/常规查看、增加轨迹—机制关联的界面、以及在同样关联上增加证据状态表达的界面,从而分别估计信息组织与证据校准的作用。

1.5.3 客观真值、任务材料与生态有效性

运行理解实验的难点之一是建立客观真值。真实项目具有生态有效性,却常缺少唯一可验证的根因;人工小程序容易精确控制,却可能把调试简化为找明显错误。AAR/AI 通过在同一系统中预设十个已知故障,同时测量召回与误报,为真值驱动评价提供了范例[4]。

AgentStepper 把真实软件开发 Agent 与预先分析的轨迹缺陷结合,既保留较真实的任务材料,又能判断参与者是否找到目标缺陷[18]。Cornelissen 等则以代表性系统和八项理解任务覆盖多种查询类型,避免用单一问题代表整体理解[33]。这些做法说明,较合适的材料不是完全合成或完全自然的二选一,而是把真实运行与经过插桩的构造案例组合使用。

具体而言,真实运行可用于检验界面在噪声、长轨迹和非理想数据中的可用性;构造案例则可有计划地操纵上下文保留、替换、裁剪、摘要和工具结果注入等机制,使每个问题具有明确答案。任务应覆盖“发生了什么”“哪条信息进入了请求”“哪个机制改变了它”“当前证据能否支持该结论”等不同层级,并在难度、轨迹长度、关键证据位置和机制类型上配平。除最终答案外,还应记录参与者打开了哪些事件、是否查看机制依据、在何处改变判断,以及是否在证据不足时主动保留未知。这样才能区分偶然答对、依赖常识猜测与真正沿证据链形成解

释。

实验还需控制学习效应和界面熟悉度。三种条件若由同一参与者依次体验，后续条件可能因熟悉任务而获益；若采用组间设计，小样本又可能受到经验差异影响。可采用拉丁方平衡条件顺序，并在正式任务前设置不计分的训练案例；同时收集 Agent 开发经验、传统调试经验和可视化素养，用于检验个体差异。Romero 等发现外部表示知识会显著影响多视图调试表现[34]，因此不能把所有错误都归因于界面。通过屏幕记录、交互日志、任务后访谈和客观答案的联合分析，可以把“界面没有提供线索”“参与者没有发现线索”和“发现后仍作出错误推断”区分开来。

表 3 Agent 运行理解研究的评价范式

评价范式	代表研究	常用结果	可回答的问题	主要局限
系统展示/案例	AutoGen Studio[25]、AgentGraph[23]、LADYBUG[28]	功能覆盖、案例完成、性能开销	系统是否可实现并接入真实轨迹	不能证明人的理解得到提升
专家评议/访谈	Graph of Trace[24]、AgentLens[22]	可用性、可理解性、需求反馈	界面是否符合工作实践	主观清晰感可能偏离客观正确性
受控调试实验	Whyline[5][6][7]、AgentStepper[18]	成功率、时间、缺陷定位、负荷	交互工具是否改善具体调试任务	样本和任务通常较小，生态有效性有限
真值驱动的理解评价	AAR/AI[4]、轨迹可视化实验[33]	召回率、精确率、正确率、用时	用户结论是否与已知事实一致	需要精心构造且难度相当的任务材料

1.6 国内外研究现状与不足

从研究脉络看，国外相关工作起步较早，并形成了由传统程序调试、分布式追踪、可视分析到 Agent 交互工具的连续积累。Whyline、信息觅食和故障定位研究提供了以用户问题组织证据、以导航行为解释调试成本的基础[5][8][15]；X-Trace、Dapper 和 Pivot Tracing 提供了跨组件关联、上下文传播与动态因果查询的方法[30][31][32]；provenance 和不确定性研究则把记录历史、语义层级、信任校准和证据边界纳入界面设计[36][37][39][40]。近年的 AGDebugger、AgentStepper、AgentGUI、AgentLens 和 AutoGen Studio 开始针对 LLM Agent 长

轨迹提供观察、下钻和干预能力[2][13][18][22][25]，研究重点正从模型输出解释转向完整运行过程。

国内及华人团队近年的研究更多集中在 Agent 轨迹结构化、Harness 诊断、自动演化与科学工作流可视化。Who&When 以大规模多 Agent 任务揭示自动故障归因的难度[3]；HarnessFix 提出面向 Harness 的数据流、控制流与实现锚点，将失败证据连接到具体运行机制[20]；AHE 从组件、经验和决策三个层面建立可观测闭环[21]；AgentLens、AgentGraph 和 Graph of Trace 探索层级时间视图、知识图谱和实时工作流图对复杂 Agent 行为的表达[22][23][24]；面向 Agent 执行溯源的综述则尝试统一检索、工具、记忆和故障诊断中的证据关系[27]。这些工作说明，国内研究已经具备从技术可观测走向交互理解的良好基础，但针对开发者认知过程的受控人因证据仍然不足。

综合来看，现有研究仍存在四方面缺口。第一，Agent 可观测工具主要围绕线性日志、时间视图或任务图组织运行事件，对“某条轨迹为何由某个 Harness 机制塑造”的显式映射支持有限。即使 HarnessFix 已经生成这种映射，其主要消费者仍是自动诊断与修复模块，而非开发者[20]。第二，工具常把原始记录、规则推导和模型生成的候选解释放在相近的视觉层级，用户难以判断证据强弱与未知边界。第三，现有评价多集中于任务成功率、系统性能、可用性或主观清晰度，较少同时测量机制理解正确性、证据定位和无依据结论。第四，许多研究一次性加入新数据、新算法和新界面，难以分离“获得额外信息”与“更好组织既有信息”的作用。

本研究据此将问题限定为：在一次 Agent 运行已经发生、且暂不要求系统自动修复的情况下，如何通过交互界面帮助开发者把轨迹中的现象关联到 Harness 机制，并在形成结论时识别证据属于实际记录、规则推导还是目前未知。研究拟先通过形成性研究分析开发者在真实运行中的问题、搜索路径和误判，再在既有 DSH Architecture 原型上增加轨迹—机制关联与证据状态表达，最后利用带有客观真值的真实运行和仪器化构造案例开展三条件受控实验。三种条件共享同一底层数据，分别呈现常规日志、轨迹—机制关联、轨迹—机制关联加证据状态，

以比较理解正确率、证据定位、无依据结论、时间、工作负荷和导航行为。该设计既继承 Whyline 以问题组织证据的思想，也吸收 Data-Frame 意义建构、分布式追踪和 provenance 研究对关系与边界的要求。

1.7 本章小结

本章围绕 Agent Harness 运行理解梳理了五条相关研究线索。其一，LLM 产品评价和 Harness 研究表明，最终表现不能解释系统下一步应修改什么，静态 Agent 规格也不能替代真实运行证据[1][20][29]。其二，传统调试与意义建构研究说明，开发者需要从可观察异常出发提出 why 与 why-not 问题，在信息拓扑中搜索，并通过证据持续修正解释框架[5][8][12][17]。其三，Agent 可观测与交互调试工具已经能够记录、浏览、下钻和干预长轨迹，但轨迹结构、机制实现与证据强度尚未被统一呈现[2][18][23]。其四，分布式追踪和 provenance 研究提供了稳定标识、因果关联、语义层级和可核查历史等技术基础，同时强调缺失记录不能被解释为事件未发生[30][31][36][37]。其五，不确定性与人因评价研究表明，界面不仅要提供更多信息，还要帮助用户校准结论，并通过具有客观真值的任务检验理解是否真正改善[4][33][39][40]。

因此，当前领域的核心矛盾已不是“能否收集 Agent 日志”，而是“能否把分散的运行证据组织成开发者可检查的轨迹—机制关联，并明确关联的证据状态”。本研究拟在这一交叉点上展开：把运行时上下文构造作为主要机制切口，把人的理解与故障定位作为主要任务，把记录、推导和未知作为关键界面变量，并通过条件隔离的受控实验检验其贡献与意义。

参考文献

- [1] VAN DER MADEN W, SADEK M, XIAO Z, et al. Results-Actionability Gap: Understanding How Practitioners Evaluate LLM Products in the Wild[C]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026: 1-17.
- [2] EPPERSON W, BANSAL G, DIBIA V, et al. Interactive Debugging and Steering of Multi-Agent AI Systems[C]//Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems. 2025: 156:1-156:15.
- [3] ZHANG S, YIN M, ZHANG J, et al. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems[C]//Proceedings of Machine Learning Research. 2025, 267: 76583-76599.
- [4] KHANNA R, DODGE J, ANDERSON A, et al. Finding AI's Faults with AAR/AI: An Empirical Study[J]. ACM Transactions on Interactive Intelligent Systems, 2022, 12(1): 1:1-1:33.
- [5] KO A J, MYERS B A. Debugging Reinvented: Asking and Answering Why and Why Not Questions about Program Behavior[C]//Proceedings of the 30th International Conference on Software Engineering. 2008: 301-310.
- [6] KO A J, MYERS B A. Designing the Whyline: A Debugging Interface for Asking Questions about Program Behavior[C]//Proceedings of the 2004 ACM Conference on Human Factors in Computing Systems. 2004: 151-158.
- [7] KO A J, MYERS B A. Finding Causes of Program Output with the Java Whyline[C]//Proceedings of the 27th ACM Conference on Human Factors in Computing Systems. 2009: 1569-1578.
- [8] LAWRENCE J, BOGART C, BURNETT M, et al. How Programmers Debug, Revisited: An Information Foraging Theory Perspective[J]. IEEE Transactions on Software Engineering, 2013, 39(2): 197-215.
- [9] RASMUSSEN J, JENSEN A. Mental Procedures in Real-Life Tasks: A Case Study of Electronic Trouble Shooting[J]. Ergonomics, 1974, 17(3): 293-307.
- [10] KLEIN G, MOON B, HOFFMAN R R. Making Sense of Sensemaking 1: Alternative Perspectives[J]. IEEE Intelligent Systems, 2006, 21(4): 70-73.
- [11] KLEIN G, MOON B, HOFFMAN R R. Making Sense of Sensemaking 2: A Macrocognitive Model[J]. IEEE Intelligent Systems, 2006, 21(5): 88-92.
- [12] KLEIN G, PHILLIPS J K, RALL E L, et al. A Data-Frame Theory of Sensemaking[M]//Expertise Out of Context. Mahwah, NJ: Lawrence Erlbaum Associates, 2007: 113-155.
- [13] ZHAO X, SOHN J, ZHENG Q, et al. AgentGUI: An Interface for Observing and Steering Long-Running AI Agents[EB/OL]. arXiv:2607.26300, 2026.
- [14] KO A J, MYERS B A, COBLENZ M J, et al. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks[J]. IEEE Transactions on Software Engineering, 2006, 32(12): 971-987.
- [15] KATZ I R, ANDERSON J R. Debugging: An Analysis of Bug-Location Strategies[J]. Human-Computer Interaction, 1987-1988, 3(4): 351-399.

- [16] STARR A, STOREY M A. Theory of Troubleshooting: The Developer's Cognitive Experience of Overcoming Confusion[J]. ACM Transactions on Software Engineering and Methodology, 2026. DOI: 10.1145/3800945.
- [17] PIROLI P, CARD S K. The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis[C]//Proceedings of the 2005 International Conference on Intelligence Analysis. 2005.
- [18] HUTTER R, PRADEL M. AgentStepper: Interactive Debugging of Software Development Agents[EB/OL]. arXiv:2602.06593, 2026.
- [19] ZHENG Y, HU Y, YU T, et al. AgentSight: System-Level Observability for AI Agents Using eBPF[EB/OL]. arXiv:2508.02736, 2025.
- [20] CHEN M, WANG J, LIU Z, et al. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws[EB/OL]. arXiv:2606.06324v2, 2026.
- [21] LIN J, LIU S, PAN C, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses[EB/OL]. arXiv:2604.25850v4, 2026.
- [22] LU J, PAN B, CHEN J, et al. AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems[J]. IEEE Transactions on Visualization and Computer Graphics, 2025, 31(8): 4182-4197.
- [23] WU Z, CHO S, VILLALOBOS C E M, et al. AgentGraph: Trace-to-Graph Platform for Interactive Analysis and Robustness Testing in Agentic AI Systems[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2026, 40(48): 41721-41723.
- [24] GAO T, LI H, LI J H, et al. Graph of Trace: Visualizing Execution Traces of Scientific Agents[C]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. 2026: 297-306.
- [25] DIBIA V, CHEN J, BANSAL G, et al. AutoGen Studio: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 2024: 72-79.
- [26] ALSAYYAD A, HUANG K Y, PAL R. AgentTrace: A Structured Logging Framework for Agent System Observability[EB/OL]. arXiv:2602.10133, 2026.
- [27] WANG Y, ZHANG J, CAI T, et al. From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents[EB/OL]. arXiv:2606.04990, 2026.
- [28] RORSETH J, GODFREY P, GOLAB L, et al. LADYBUG: an LLM Agent DeBUGger for Data-Driven Applications[C]//Proceedings of the 28th International Conference on Extending Database Technology. 2025: 1082-1085.
- [29] AMINI S, BENAJIBA Y, BERNARDIS C, et al. Open Agent Specification: Enabling Cross-Framework Comparison of AI Agents[C]//Proceedings of the ACM Conference on AI and Agentic Systems. 2026: 404-418.
- [30] FONSECA R, PORTER G, KATZ R H, et al. X-Trace: A Pervasive Network Tracing Framework[C]//Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation. 2007: 271-284.
- [31] MACE J, ROELKE R, FONSECA R. Pivot Tracing: Dynamic Causal Monitoring for Distributed

Systems[C]//Proceedings of the 25th ACM Symposium on Operating Systems Principles. 2015: 378-393.

- [32] SIGELMAN B H, BARROSO L A, BURROWS M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure[R]. Google Technical Report dapper-2010-1, 2010.
- [33] CORNELISSEN B, ZAIDMAN A, VAN DEURSEN A. A Controlled Experiment for Program Comprehension through Trace Visualization[J]. IEEE Transactions on Software Engineering, 2011, 37(3): 341-355.
- [34] ROMERO P, DU BOULAY B, COX R, et al. Debugging Strategies and Tactics in a Multi-Representation Software Environment[J]. International Journal of Human-Computer Studies, 2007, 65(12): 992-1009.
- [35] PARNIN C, ORSO A. Are Automated Debugging Techniques Actually Helping Programmers?[C]//Proceedings of the 20th International Symposium on Software Testing and Analysis. 2011: 199-209.
- [36] RAGAN E D, ENDERT A, SANYAL J, et al. Characterizing Provenance in Visualization and Data Analysis: An Organizational Framework of Provenance Types and Purposes[J]. IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 31-40.
- [37] XU K, ATTFIELD S, JANKUN-KELLY T J, et al. Analytic Provenance for Sensemaking: A Research Agenda[J]. IEEE Computer Graphics and Applications, 2015, 35(3): 56-64.
- [38] KADIVAR N, CHEN V, DUNSMUIR D, et al. Capturing and Supporting the Analysis Process[C]//Proceedings of the IEEE Symposium on Visual Analytics Science and Technology. 2009: 131-138.
- [39] SACHA D, SENARATNE H, KWON B C, et al. The Role of Uncertainty, Awareness, and Trust in Visual Analytics[J]. IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 240-249.
- [40] HULLMAN J. Why Authors Don't Visualize Uncertainty[J]. IEEE Transactions on Visualization and Computer Graphics, 2020, 26(1): 130-139.

三、选题报告

2 文献综述

大语言模型（Large Language Model, LLM）Agent 已由单轮问答扩展为能够调用工具、访问环境、维护记忆并持续执行任务的复合系统。此类系统的行为不仅由基础模型决定，还受到提示组织、上下文装配、工具接口、控制循环、状态管理、错误处理和验证规则等运行基础设施的共同塑造。本文将这些围绕模型、负责把一次任务组织成可执行过程的基础设施统称为 **Agent Harness**。对开发者而言，真正困难的问题已不只是判断任务是否成功，而是解释某一运行现象如何产生、哪些轨迹证据与之相关，以及这些证据如何连接到具体的 **Harness** 机制。围绕这一问题，现有研究主要分布在 LLM 产品评价、程序调试与意义建构、Agent 可观测与交互调试、分布式追踪与分析溯源、不确定性表达及人因评价等领域。下面依次梳理这些研究，并据此界定本研究的问题边界。

2.1 Agent Harness 与运行理解的概念边界

2.1.1 从模型评价到系统运行评价

早期 LLM 研究通常把模型能力作为主要评价对象，以准确率、胜率或基准任务得分衡量效果。然而，真实产品中的输出是模型、提示、检索、界面、业务规则与人员协作共同作用的结果。van der Maden 等对 19 名 LLM 产品实践者的访谈表明，17 名受访者都经历过“结果—可行动性鸿沟”：现有评价能够揭示产品表现，却难以指出下一步应修改哪个系统部分^[1]。这一发现说明，当评价对象从孤立模型转向完整 Agent 系统时，最终分数无法替代运行机制层面的解释。Khanna 等提出的 AAR/AI 也表明，相较于总体性能指标，把问题定位到具体决策与内部预测能够帮助使用者发现更多 AI 故障；在具有已知真值的任务中，该方法提高了问题召回率和识别故障的可能性，但精确率仍然有限^[2]。由此可见，运行理解既需要比结果指标更细粒度的证据，也必须防止把局部异常过早解释为已确定的根因。

Harness 概念进一步揭示了系统评价为何不能等同于模型评价。Chen 等把 Harness 划分为执行环境与沙箱、工具接口、上下文与记忆、生命周期与编排、可观测、验证评估、治理安全等层，并指出失败轨迹中的决定性证据往往散落在多个层次^[3]。Lin 等则把系统提示、工具、中间件、技能、子 Agent、长期记忆和配置视为可编辑 Harness 组件，强调高性能 Agent 依赖的不只是提示词，而是组件之间可被追踪、比较和回滚的工程结构^[4]。Amini 等在四种运行时上执行相同的 Agent 声明，仍观察到准确率、延迟和执行行为的明显差异，说明静态规格相同并不保证运行语义相同^[5]。因此，本研究中的“Harness 运行理解”并非阅读配置文件，也不是概括 Agent 的自然语言理由，而是把实际执行过程与塑造该过程的运行机制联系起来。

2.1.2 运行轨迹、机制关联与故障归因

Agent 运行轨迹通常包含模型请求与响应、工具调用与结果、文件或环境变化、Agent 间消息、重试与终止等事件。轨迹记录回答“发生了什么”，Harness 机制回答“系统以什么规则组织了这些事件”，两者之间的关联才可能支持“为什么出现该现象”的解释。Zhang 等把多 Agent 故障归因分解为责任 Agent 和责任步骤两个维度，在 127 个系统、184 个归因任务上的结果显示，即使先进方法对责任 Agent 的识别准确率达到 53.5%，对具体步骤的准确率也只有 14.2%^[6]。这说明长轨迹中的“谁”和“何时”本身就难以可靠确定，更不能把自动归因结果直接当作事实呈现。

同时，运行理解与自动修复应当区分。HarnessFix 把原始轨迹与 Harness 工件编译为中间表示，通过数据流、控制流和实现锚点把失败步骤映射到责任层，再生成并验证修补方案^[3]；AHE 则通过组件可观测、经验可观测和决策可观测构造 Harness 自动演化闭环^[4]。两类工作证明了轨迹—机制关联在技术上可被计算，也显示这种关联能够支持系统改进。但它们主要以任务性能、修复收益和跨模型迁移为目标，尚未回答开发者如何检查关联证据、如何区分直接记录与算法推断，以及界面呈现是否会造成新的误解。因此，本文所说的运

行理解以人的证据判断和故障定位为中心，不以自动生成修复方案为首要目标。

2.1.3 上下文构造作为关键研究切口

在 Agent Harness 的诸多机制中，上下文构造具有连接模型、工具和历史状态的枢纽作用。一次请求实际接收哪些系统指令、历史消息、工具结果与检索材料，哪些内容被保留、替换、裁剪或摘要，直接改变模型下一步可见的信息。AgentTrace 把上下文事件与认知事件、操作事件并列为三类结构化日志，说明仅记录工具调用无法重建 Agent 当时的输入条件^[7]。AgentSight 从系统边界同时观测模型侧语义意图和内核侧行为后发现，高层意图与低层效果之间存在明显语义鸿沟^[8]。Open Agent Specification 的跨框架实验进一步表明，即使共享声明结构，不同运行时对提示和工具的处理仍会造成行为差异^[5]。这些研究共同支持以“信息在何处进入请求、如何被转换、是否仍可被后续步骤看见”为核心分析对象。相较于覆盖所有 Harness 层，这一切口既与轨迹证据直接相连，也便于构造具有客观真值的实验案例。

2.2 开发者调试与意义建构的认知基础

2.2.1 从异常现象到 why 与 why-not 问题

传统调试研究指出，开发者通常先观察到异常输出，再逐步提出关于原因的假设。Katz 和 Anderson 将“定位错误”与“判断、修复错误”区分开来，发现即便程序员有能力修复已经指出的错误行，也未必能从行为表现中有效找到该位置^[9]。这一区分对 Agent 运行理解尤为重要：用户能够看懂某条工具报错，并不意味着能够发现导致该报错的上下文裁剪或控制分支。

Ko 和 Myers 围绕 why 与 why-not 问题形成的一系列研究，把调试入口从“先猜代码位置”改为“直接询问可见行为为什么发生或为什么没有发生”。Whyline 界面允许用户针对程序输出提出问题，再由系统生成相关运行时因果链，早期研

究显示其能够显著缩短问题定位时间^[10]。后续 Java Whyline 扩展到真实 Java 程序，在受控实验中提高了成功完成调试任务的比例，并减少了定位时间^[11]。在此基础上，“Debugging Reinvented”进一步论证：如果工具能围绕开发者实际拥有的疑问组织证据，就能减少断点与打印语句造成的无关搜索^[12]。这些研究的核心价值不在于复用某一种传统调试器控件，而在于把界面组织单位从原始事件转为用户可提出的问题，并保留问题到证据之间可检查的联系。

Agent 系统已经开始吸收这一思路。AGDebugger 面向多 Agent 消息轨迹提供暂停、重置和修改消息等交互，使开发者能够在错误传播前探索替代路径；形成性访谈和 14 人实验表明，交互调试有助于定位问题，但参与者在需要正确选择干预位置和方式时仍频繁失败^[13]。AgentStepper 把断点、单步执行、中间状态检查和提示或工具调用编辑引入软件开发 Agent。其 12 人研究中，参与者识别 Agent 实现缺陷的成功率由 17% 提高到 60%，挫败感由 5.4 降至 2.4，但一般轨迹解释成绩的提升并不明显^[14]。这意味着“能操控轨迹”与“能形成正确解释”不是同一件事，交互功能必须与证据结构和机制关联共同设计。

2.2.2 信息觅食、层级搜索与跨表示协调

调试过程也可被理解为在信息空间中寻找具有价值的线索。Lawrance 等以信息觅食理论重新分析专业程序员的调试行为，发现开发者根据界面线索形成“信息气味”，并在文件、方法和调用关系组成的“信息拓扑”中决定继续追踪还是转换路径^[15]。Ko 等对软件维护任务的观察显示，参与者平均约 35% 的有效时间用于导航、关联和收集信息，说明工具切换和线索组织本身就是理解成本的重要来源^[16]。

Rasmussen 和 Jensen 对真实电子故障排查的研究则表明，熟练人员会在系统、功能单元和部件之间进行层级搜索，并在信息收益、测量成本与认知负荷之间折衷^[17]。这些结果提示，Agent 调试界面不应要求用户从头顺序阅读完整日志，而应提供从异常症状下钻到局部事件、再上溯到机制的可导航拓扑。

但增加视图不一定带来更好理解。Romero 等研究同时呈现代码、输出与图形表示的调试环境，发现成功表现不仅依赖编程经验，也依赖用户理解外部表示并协调不同表示的能力；较有经验者能够根据任务调整策略，经验较少者却可能承受额外切换成本^[18]。Cornelissen 等对执行轨迹可视化的受控实验显示，在八项程序理解任务中，轨迹可视化组用时减少 22%，正确率提高 43%^[19]，证明结构化轨迹具有潜在价值；然而该结果并不意味着任意可视化都有效，因为实验中的收益来自任务、视图和交互之间的具体匹配。对本研究而言，时间轴、关系图、机制代码与证据详情必须通过稳定标识和明确选中状态相互对应，避免用户在多个视图之间重新寻找同一事件。

2.2.3 Data - Frame 意义建构与困惑消解

运行理解不是一次性读取事实，而是持续构造和修正解释框架。Klein 等对不同 sensemaking 传统的梳理指出，复杂工作中的理解包含联系信息、解释意外、维持连贯叙事并支持下一步行动等活动^[20]。其宏观认知模型强调，既有框架会引导人关注特定数据，新的数据又可能强化、质疑或替换该框架^[21]。Data - Frame 理论进一步把这一过程描述为框架与数据之间的双向循环：人利用框架解释线索，当异常无法纳入当前框架时，便进行质疑、比较和重构^[22]。Agent 运行中的一次异常工具结果、缺失消息或意外终止，正是触发框架更新的关键线索。

Pirolli 和 Card 通过认知任务分析把意义建构划分为信息觅食循环与意义建构循环：前者负责搜索、筛选和组织材料，后者负责建立证据结构、生成假设并形成结论^[23]。这一模型说明，支持运行理解的界面既要降低查找证据的成本，也要允许用户保留和比较暂时解释。Kadivar 等的 CzSaw 系统通过记录分析历史、依赖关系、分支和回放，使分析者能够返回旧状态、探索替代路径并复用已有过程^[24]。Xu 等进一步指出，低层点击容易自动捕获，高层任务、意图和推理却难以准确推断；分析溯源若把自动推断的高层语义伪装成原始记录，反而会破坏反思与协作^[25]。

Starr 和 Storey 从 27 名开发者访谈中总结了故障排查中的困惑、试探、观察和恢复过程，指出困惑会占用注意与工作记忆，并随时间转化为疲劳^[26]。这为 Agent 界面的设计提供了重要人因边界：系统不应以更多字段和图形制造“可观测即理解”的假象，而应围绕当前问题逐步揭示关联信息；同时，应保留用户回退、比较和修正判断的空间。换言之，理想界面不是一次给出完整答案，而是帮助开发者在证据约束下形成、检查并更新自己的 Data - Frame。

2.3 Agent 轨迹可观测与交互调试研究进展

2.3.1 从结构化日志到多粒度轨迹

Agent 可观测研究首先解决“记录什么”。AgentTrace 提出操作、认知和上下文三个日志平面，并通过统一的 trace id 与 span id 组织模型交互、环境动作和上下文变化^[7]。该框架把运行日志从运维附属物提升为安全、复现和问责的基础，但目前主要贡献是日志模式，尚未验证不同呈现方式对开发者理解的影响。AgentSight 采用 eBPF 在不修改 Agent 代码的情况下捕捉模型通信和系统事件，通过边界追踪关联高层意图与低层效果，开销低于 3%^[8]。它拓展了可观测的覆盖面，也带来新的证据问题：系统调用是直接观测到的，语义意图及二者关系可能包含模型分析或规则推断，界面必须让用户识别这种差异。

传统分布式追踪为 Agent 轨迹组织提供了成熟概念。X-Trace 通过显式传播任务标识，把跨应用、协议和管理域的事件重建为任务树^[27]；Dapper 以 trace 与 span 表达一次请求的分布式路径，并以通用库插桩和采样在覆盖率、透明度与运行开销之间折衷^[28]。这些系统说明，可靠关联依赖稳定标识和因果上下文传播，而不是事后仅凭时间邻近猜测。但采样、未插桩组件或标识传播中断都会造成空白，因此“轨迹中没有记录”不能直接推出“事件没有发生”。Pivot Tracing 进一步引入 happened-before join，使用户能够在运行时提

出跨组件查询并依据先发生关系连接事件^[29]。它展示了规则推导关联的价值，也提示推导结果应附带所用规则与输入证据。

2.3.2 面向人的观察、调试与干预工具

现有 Agent 工具大致形成三类界面。第一类以时间和层级结构帮助用户浏览长轨迹。AgentLens 从原始事件构建行为结构，以层级时间视图展示多 Agent 行为演化，并允许从总体模式下钻到具体行为及候选原因^[30]。

AgentGUI 同时呈现任务、轨迹、产物和成本，并提供人工与自动纠偏；其实验报告轨迹元素识别速度提高 38%，

识别准确率由 80% 提升至 93%^[31]。这些工具证明了概览—细节协同的价值，但候选原因或自动管理建议仍可能包含推断，若缺少证据强度标记，用户容易把视觉邻近或系统推荐理解为确定因果。

长轨迹界面还面临“压缩与保真”的矛盾。概览必须把成百上千个事件归纳为可浏览单元，否则用户仍会陷入线性阅读；但摘要又可能删除决定性参数、异常返回值或消息边界。AgentLens 通过层级聚合压缩事件，Graph of Trace 通过有向图突出阶段和依赖，AgentGUI 通过任务和产物视图减少终端噪声^[30-32]。这些方法都在降低搜索成本，却没有彻底解决摘要与原始记录的对应问题。对运行理解而言，任何被压缩的节点都应能展开到原始事件，摘要文本应被视为一种派生表示，而不是原始证据本身；若摘要由 LLM 生成，还应说明生成范围和遗漏风险。该要求与 Whyline 的设计原则一致：界面可以替用户缩小搜索空间，但必须允许用户检查系统据以回答问题的实际运行证据^[11-12]。

第二类以结构图重建任务与依赖关系。AgentGraph 把 Agent、任务、工具、输入输出和人员转为知识图谱，每个图元素都链接到精确轨迹片段，因而具有较好的可核验性^[33]。Graph of Trace 把科学 Agent 的执行事件组织成持续更新的有向图，领域专家认为结构化轨迹有助于理解 workflow、检查中间步骤和后续协作^[32]。两者均说明图结构适合表达长链依赖，但现有评价多是系统展

示、使用情景或主观反馈，尚未充分测量用户能否准确区分已记录关系、算法重建关系与未知关系。

第三类将观察与干预结合。AutoGen Studio 以声明式 JSON 描述 Agent 与 workflow，通过拖拽界面、运行游乐场和消息查看支持多 Agent 原型构建与调试^[34]；LADYBUG 允许用户检查每一步输入输出、在任意步骤干预并只重算受影响部分，同时利用 LLM 提出候选错误与修正^[35]；AGDebugger 与 AgentStepper 则进一步支持暂停、回退、单步和现场编辑^[13-14]。这类工具降低了试验替代路径的成本，却也把“观察”“判断”和“改变系统”紧密耦合。若用户在尚未确认原因时直接改写轨迹，新的结果可能被误当作对原假设的验证。因此，面向运行理解的界面应先支持证据定位与机制核对，再把干预作为可选的后续活动。

2.3.3 自动归因、修复与人的理解之间的断层

Agent 故障诊断研究正在从“显示轨迹”走向“解释轨迹”。Who&When 尝试自动判断责任 Agent 与步骤，但步骤级准确率仍低，表明归因模型本身需要被审查^[6]。

HarnessFix 将轨迹片段映射到 Harness 层与实现工件，并在四类基准上通过针对性修复获得性能提升^[3]；AHE 把每次 Harness 编辑与预期结果写成可证伪契约，并用下一轮任务结果验证^[4]。这些方法的共同贡献是：它们不再把失败轨迹仅视为负向奖励，而把轨迹视为定位机制问题的结构化证据。

不过，自动系统的正确率或修复收益不能直接证明开发者获得了正确理解。Parnin 和 Orso 对自动故障定位工具的研究指出，算法把错误语句排在前列，并不意味着程序员能够理解根因；真实调试还需要沿数据和控制依赖导航、检查状态并通过重跑检验假设^[36]。这一批评同样适用于 Agent 归因：一个“责任步骤”标签若没

表 2.1 代表性 Agent 运行可观测与交互调试系统比较

代表工作	主要表示	核心交互	轨迹—机制关	证据状态呈	人因评价特现
			联	点	

AGDebugger ^[13]	多 ¹ Agent 消 迹 编辑消息	暂停、重置、息 主要关联消息 与 Agent 角色	未显式区分	形成性访谈与 14 人实验, 关注定位和 干预
AgentGUI ^[31]	任务、轨迹、产 物与成本视图	观察、人工/自 动纠偏	可定位轨迹元 素, 机制映射 有限	未系统区分 测量时间与正 确率
AgentStepper ^[14]	结构化会话 ¹ 与代码变化	断点、单步、 编辑、检查状 态	关联 Agent 动 作与实现执行	未显式区分 12 人对照研 究, 测量缺 陷识别和负 荷
AgentLens ^[30]	层级时间视 图	概览、筛选、 下钻、原因追 踪	部分原因由算 法挖掘	推断边界不突 出 使用情景与用 户研究
AgentGraph ^[33]	交互知识图 谱	图浏览、回到 原始轨迹	图元素链接精 确 trace span	可核验, 但 状态分类有 限 系统展示为 主
Graph of Trace ^[32]	实时有向工作 流图	过程检查与阶 段回顾	依赖由监测与 解析模块重建	未独立编码 领域专家主观评 价
HarnessFix ^[3]	Harness- aware 中间表	自动归因与 修复示	数据流、控制 流、实现锚点	面向机器诊 和修复效果为主 以基准性能断

有展示其数据来源、关联规则与缺失信息，可能提高操作速度，却也可能诱发过度确定。由此形成当前研究的关键断层：技术系统已能产生丰富的关联和候选解释，人机交互研究却较少验证这些解释如何被开发者理解、误解和校准。

从可行性角度看，人机界面与自动修复系统还具有不同的输出要求。自动修复可以在大量候选中搜索，并以回归测试筛选有效补丁；开发者界面则必须让人理解当前判断为何成立，以及不成立时应从哪里继续调查。AHE 把一次编辑前的预测与下一轮结果绑定，强调“修改—预期—验证”的可审计链^[4]；HarnessFix 使用实现锚点限制修复范围，避免从局部现象跳到宽泛改动^[3]。若把这种思想转向人机交互，界面应支持开发者把每项判断写成可验证主张，并指出它由哪些记录和规则支持、哪些替代解释尚未排除。这样，运行理

解的产物就不只是一个根因标签，而是能够被同事复核、被后续运行证伪或强化的证据化解释。

2.4 轨迹—机制关联的证据基础

2.4.1 provenance 的对象与用途

Provenance 研究为“从哪里来、经过什么过程、为何形成当前结果”提供了系统

概念。Ragan 等梳理可视化与数据分析中的 provenance，将其区分为数据、可视化、交互、洞见和理由等类型，并强调记录类型必须与使用目的相匹配[37]。如果目标是复现运行，保存参数、输入和执行环境可能已经足够；如果目标是解释开发者为何形成某一判断，还需要假设、注释和证据选择过程。Wang 等针对 LLM Agent 进一步提出统一的执行溯源框架，把检索证据、工具结果、记忆项、环境观察、中间声明、动作与最终答案组织为带类型的关系，并指出现有工作仍缺少统一模式、语义级 provenance 和面向恢复过程的评价[38]。

分析溯源还揭示了语义层级问题。Xu 等提出从低层事件、分析动作、子任务到总体任务的层次模型，指出语义越高，越难仅靠系统日志可靠推断[25]。CzSaw 通过自动记录交互历史并允许用户编辑、分支和重放，在机器记录与人的解释之间建立互补关系[24]。对 Agent Harness 而言，工具返回文本、请求长度、消息删除等可以直接记录；“这条结果因上下文裁剪而未被模型使用”可能需要结合数据流规则推导；“模型为何忽略仍然可见的信息”则可能超出当前证据能力。界面若把三者以同样样式呈现，就会掩盖语义跃迁和推断风险。

2.4.2 记录、推导与未知的证据状态

基于分布式追踪、Agent 可观测和 provenance 研究，可以把面向运行理解的证据划分为三类状态。第一类是“实际记录到”，指系统在运行时直接捕获且能够回到原始事件的事实，例如某次模型请求的消息列表、某个工具调用参数、某条文件写入事件。第二类是“按规则推导”，指依据显式规则把多个已记录事实连接起来，例如通过 trace id 重建父子 span，通过 happened-before 关系确定先后依赖，或通过 Harness 代码中的构造函数把输入字段映射到请求片段^[27-29]。第三类是“目前无法知道”，指由于未插桩、采样、状态未持久化或模型内部过程不可观测，现有证据不能支持肯定结论。

这种分类并非简单的可信度高、中、低。直接记录也可能不完整或来自错误时钟；规则推导可以高度可靠，但必须暴露规则和前提；未知则是一种必要结论，而不是系统失败。Sacha 等指出，不确定性会从数据、模型、可视化到人的解释逐层传播，用户对系统的信任取决于是否意识到这些不确定性^[39]。Hullman 对 90 名可视化作者的问卷和 13 名设计者的访谈发现，作者常因担心复杂度、说服力下降和读者认知负担而省略不确定性，尽管他们同时承认不确定性有价值^[40]。这揭示了一个重要设计矛盾：证据状态不能被隐藏，但也不宜用密集概率或警告淹没主要任务。更合适的方式是以稳定、可解释的状态标识呈现，并允许用户查看状态依据。

证据状态的呈现还应避免把“未知”处理成视觉上的次要信息。调试活动具有强烈的闭合倾向：当界面给出一条流畅的候选原因或醒目的高亮路径时，用户可能停止寻找反证。Data - Frame 理论认为，异常线索是触发框架重构的重要条件^[22]；若未知信息被折叠，用户将难以发现当前框架不能解释的部分。因而，状态设计至少需要完成三项功能：让用户一眼区分事实与推导；在推导处提供可展开的规则、输入和反例条件；在未知处说明缺失信号及可能的补证方式。对于无法通过本次运行补证的问题，界面也应允许用户保留“尚不能判断”，而不是强迫选择单一原因。这里的目标并非削弱信任，而是把信任从对系统权威的笼统接受转化为对具体证据链的校准。

表 2.2 Agent Harness 运行理解中的证据状态

证据状态	判定依据	典型例子	界面需要回答的 主要风险问题
实际记录到	有原始事件、时 戳与稳定标识，	请求中实际包含 记录位于何处，	把“记录到结间 果”误解为 “已可返回源记录 实返回值；文件 哪次运行 证明原因” 写入
按规则推导	由已记录事实和 由父子 span 还	使用了什么规则， 把算法推断误 当公开规则计算， 原调用链；由代 前提是否满足， 作直接观察 能够说明输入与 码锚点推得上下 能否复算规则 文来源	
目前无法知	必需信号缺失或 未保存的中间上 缺少什么，是否 用时间邻近、相 道 超出系统可观测 下文；模型内部 能通过补充插桩 关性或语言流畅 边界 为何选择某动作 确认性填补空白		

2.4.3 从可追溯关系到可辩护结论

轨迹—机制关联的价值不只在于生成更多连线，而在于缩短从现象到可辩护结论的路径。X-Trace 和 Dapper 说明，跨组件关联若要稳定，需要在执行过程中传播上下文^[27-28]；Pivot Tracing 说明，事前没有记录的指标可通过动态插桩和关系查询补充，但查询结果仍依赖明确的因果顺序^[29]；AgentGraph 通过让图元素返回精确轨迹片段，提高了关系的可核查性^[33]；HarnessFix 再向前一步，把运行步骤连接到

Harness 工件和修复算子^[3]。这些工作共同形成三种关联证据：信息从哪里流入和流向何处的数据流证据，哪个分支、重试或终止规则被触发的控制流证据，以及哪个配置项、函数或文件实现该机制的实现锚点。

本研究因此不把关系图中的“边”视为同质对象。界面需要说明每条边表达的是观测到的数据传递、规则推导的控制依赖，还是面向实现的静态映射；

同时还要允许用户从边回到相关轨迹与代码位置。只有这样，用户才能判断结论的适用范围。

例如，“工具结果出现在日志中”只能证明结果产生过；若请求快照显示该结果未进入下一次模型输入，才可进一步判断上下文传递发生中断；若快照缺失，则应保留未知状态，而不是仅凭后续输出推断结果被丢弃。

2.5 现有研究的评价方法

2.5.1 从系统展示到受控理解任务

Agent 可观测研究的评价大致分为三类。第一类是系统展示和使用情景，常用于验证能否接入真实框架并呈现复杂轨迹，例如 AutoGen Studio、AgentGraph、LADYBUG 和 AgentTrace^[7,33-35]。这类研究能够说明功能可实现，却难以判断开发

者是否因此形成更准确的机制解释。第二类是专家访谈和主观反馈，例如 Graph of

Trace 邀请不同学科专家评价可理解性和可用性^[32]，AgentLens 结合使用情景与用户研究考察行为分析^[30]。它们适合发现 workflows 和界面问题，但主观“觉得清楚”可能与客观正确率不一致。

第三类是具有对照条件和客观任务的实验。Whyline 系列使用预设故障与可判断任务完成标准比较调试表现^[10-12]；轨迹可视化实验比较常规 IDE 与增加可视化后的完成时间和正确率^[19]；AAR/AI 以十个已知故障作为真值，分别测量召回率与精确率^[2]；AgentGUI 和 AgentStepper 分别测量轨迹元素识别、缺陷发现、时间与工作负荷^[14,31]。这类设计更接近本研究需要，因为“运行理解”必须通过参与者回答与客观运行机制之间的吻合程度来评价，而不能只依赖可用性量表。

2.5.2 指标选择与实验条件隔离

现有研究提示，运行理解至少包含四类可测结果。第一是理解正确性，包括对事件顺序、数据来源、机制作用和故障位置的判断；第二是证据定位能力，包括找到关键轨迹片段、机制工件和二者关系的时间与成功率；第三是结论校准，包括无依据断言、把推导说成事实、在证据不足时拒绝判断等行为；第四是过程与负担，包括完成时间、导航路径、视图切换、NASA-TLX 及主观信任。只报告任务性能会遗漏人的代价，只报告满意度又无法证明理解正确。

此外，实验需要隔离“增加了什么信息”和“如何组织相同信息”的效应。Parnin 和 Orso 指出，自动调试研究常把算法排名质量等同于对程序员的帮助^[36]；Romero 等显示，多种表示的加入可能同时改变信息量和协调负担^[18]；不确定性研究则说明，额外标记既可能校准信任，也可能增加认知成本^[39-40]。因此，如果实验组同时获得更多底层日志、更强关联算法和更丰富界面，就无法判断收益来自哪一部分。较稳妥的设计是使用共享数据底座，依次比较原始日志/常规查看、增加轨迹—机制关联的界面、以及在同样关联上增加证据状态表达的界面，从而分别估计信息组织与证据校准的作用。

2.5.3 客观真值、任务材料与生态有效性

运行理解实验的难点之一是建立客观真值。真实项目具有生态有效性，却常缺少唯一可验证的根因；人工小程序容易精确控制，却可能把调试简化为找明显错误。

AAR/AI 通过在同一系统中预设十个已知故障，同时测量召回与误报，为真值驱动评

价提供了范例^[2]。AgentStepper 把真实软件开发 Agent 与预先分析的轨迹缺陷结合，既保留较真实的任务材料，又能判断参与者是否找到目标缺陷^[14]。

Cornelissen 等则以代表性系统和八项理解任务覆盖多种查询类型，避免用单一问题代表整体理解^[19]。

这些做法说明，较合适的材料不是完全合成或完全自然的二选一，而是把真实运行与经过插桩的构造案例组合使用。

具体而言，真实运行可用于检验界面在噪声、长轨迹和非理想数据中的可用性；构造案例则可有计划地操纵上下文保留、替换、裁剪、摘要和工具结果注入等机制，使每个问题具有明确答案。任务应覆盖“发生了什么”“哪条信息进入了请求”“哪个机制改变了它”“当前证据能否支持该结论”等不同层级，并在难度、轨迹长度、关键证据位置和机制类型上配平。除最终答案外，还应记录参与者打开了哪些事件、是否查看机制依据、在何处改变判断，以及是否在证据不足时主动保留未知。这样才能区分偶然答对、依赖常识猜测与真正沿证据链形成解释。

实验还需控制学习效应和界面熟悉度。三种条件若由同一参与者依次体验，后续条件可能因熟悉任务而获益；若采用组间设计，小样本又可能受到经验差异影响。可采用拉丁方平衡条件顺序，并在正式任务前设置不计分的训练案例；同时收集 Agent 开发经验、传统调试经验和可视化素养，用于检验个体差异。Romero 等发现外部表示知识会显著影响多视图调试表现^[18]，因此不能把所有错误都归因于界面。

通过屏幕记录、交互日志、任务后访谈和客观答案的联合分析，可以把“界面没有提供线索”“参与者没有发现线索”和“发现后仍作出错误推断”区分开来。

表 2.3 Agent 运行理解研究的评价范式

评价范式	代表研究	常用结果	可回答的问题	主要局限
系统展示/案例	AutoGen Studio ^[34] 、AgentGraph ^[33] 、LADYBUG ^[35]	功能覆盖、案例完成、性能开销	系统是否可实现并接入真实轨迹	不能证明人的理解得到提升
专家评议/访谈	Graph of Trace ^[32] 、AgentLens ^[30]	可用性、可理解性、需求反馈	界面是否符合工作实践	主观清晰感可能偏离客观正确性

受控调试实验	Whyline ^[10-12] 、AgentStepper ^[14]	成功率、时间、缺陷定位、负荷	交互工具是否改善具体调试任务	样本和任务通常较小，生态有效性有限
真值驱动的理解评价	AAR/AI ^[2] 、轨迹化实验 ^[19]	召回率、精确率、用户结论是否与已知事实一致	正确率、用时	需要精心构造且难度相当的任务材料

2.6 国内外研究现状与不足

从研究脉络看，国外相关工作起步较早，并形成了由传统程序调试、分布式追踪、可视分析到 Agent 交互工具的连续积累。Whyline、信息觅食和故障定位研究提供了以用户问题组织证据、以导航行为解释调试成本的基础

^[9,12,15]；X-Trace、

Dapper 和 Pivot Tracing 提供了跨组件关联、上下文传播与动态因果查询的方法^[27-29]；

provenance 和不确定性研究则把记录历史、语义层级、信任校准和证据边界纳入界面

设计^[25,37,39-40]。近年的 AGDebugger、AgentStepper、AgentGUI、AgentLens 和 AutoGen

Studio 开始针对 LLM Agent 长轨迹提供观察、下钻和干预能力^[13-14,30-31,34]，研究重点正从模型输出解释转向完整运行过程。

国内及华人团队近年的研究更多集中在 Agent 轨迹结构化、Harness 诊断、自动演化与科学工作流可视化。Who&When 以大规模多 Agent 任务揭示自动故障归因的难度^[6]；HarnessFix 提出面向 Harness 的数据流、控制流与实现锚点，将失败证据连接到具体运行机制^[3]；AHE 从组件、经验和决策三个层面建立可观测闭环^[4]；

AgentLens、AgentGraph 和 Graph of Trace 探索层级时间视图、知识图谱和实时工作流图对复杂 Agent 行为的表达^[30,32-33]；面向 Agent 执行溯源的综述则尝试统一检索、工具、记忆和故障诊断中的证据关系^[38]。这些工作说明，国内

研究已经具备从技术可观测走向交互理解的良好基础，但针对开发者认知过程的受控人因证据仍然不足。

综合来看，现有研究仍存在四方面缺口。第一，Agent 可观测工具主要围绕线性日志、时间视图或任务图组织运行事件，对“某条轨迹为何由某个 Harness 机制塑造”的显式映射支持有限。即使 HarnessFix 已经生成这种映射，其主要消费者仍是自动诊断与修复模块，而非开发者^[3]。第二，工具常把原始记录、规则推导和模型生成的候选解释放在相近的视觉层级，用户难以判断证据强弱与未知边界。第三，现有评价多集中于任务成功率、系统性能、可用性或主观清晰度，较少同时测量机制理解正确性、证据定位和无依据结论。第四，许多研究一次性加入新数据、新算法和新界面，难以分离“获得额外信息”与“更好组织既有信息”的作用。

本研究据此将问题限定为：在一次 Agent 运行已经发生、且暂不要求系统自动修复的情况下，如何通过交互界面帮助开发者把轨迹中的现象关联到 Harness 机制，并在形成结论时识别证据属于实际记录、规则推导还是目前未知。研究拟先通过形成性研究分析开发者在真实运行中的问题、搜索路径和误判，再在既有 DSH Architecture 原型上增加轨迹—机制关联与证据状态表达，最后利用带有客观真值的真实运行和仪器化构造案例开展三条件受控实验。三种条件共享同一底层数据，分别呈现常规日志、轨迹—机制关联、轨迹—机制关联加证据状态，以比较理解正确率、证据定位、无依据结论、时间、工作负荷和导航行为。该设计既继承 Whyline 以问题组织证据的思想，也吸收 Data - Frame 意义建构、分布式追踪和 provenance 研究对关系与边界的要求。

2.7 本章小结

本章围绕 Agent Harness 运行理解梳理了五条相关研究线索。其一，LLM 产品评价和 Harness 研究表明，最终表现不能解释系统下一步应修改什么，静态 Agent 规格也不能替代真实运行证据^[1,3,5]。其二，传统调试与意义建构研究说明，开发者需要从可观察异常出发提出 why 与 why-not 问题，在信息拓扑

中搜索，并通过证据持续修正解释框架^[12,15,22-23]。其三，Agent 可观测与交互调试工具已经能够记录、浏览、下钻和干预长轨迹，但轨迹结构、机制实现与证据强度尚未被统一呈现^[13-14,33]。其四，分布式追踪和 **provenance** 研究提供了稳定标识、因果关联、语义层级和可核查历史等技术基础，同时强调缺失记录不能被解释为事件未发生^[25,27,29,37]。其五，不确定性与人因评价研究表明，界面不仅要提供更多信息，还要帮助用户校准结论，并通过具有客观真值的任务检验理解是否真正改善^[2,19,39-40]。

因此，当前领域的核心矛盾已不是“能否收集 Agent 日志”，而是“能否把分散的运行证据组织成开发者可检查的轨迹—机制关联，并明确关联的证据状态”。本研究拟在这一交叉点上展开：把运行时上下文构造作为主要机制切口，把人的理解与故障定位作为主要任务，把记录、推导和未知作为关键界面变量，并通过条件隔离的受控实验检验其贡献与意义。

3 选题报告

3.1 选题依据

3.1.1 研究来源

本课题来源于前期 Agent 运行可观测原型实践与相关文献研究。在原型实践中，运行日志、时间线和结构视图已经能够呈现模型消息、工具调用、请求信息与部分上下文来源，但开发者仍需在多个视图之间自行判断某一运行现象由哪个 Harness 机制塑造、当前判断有哪些证据支持。该问题构成了本研究的直接来源。

3.1.2 研究背景

大语言模型 Agent 正由单轮问答工具转向能够调用外部工具、访问工作环境、维护上下文并持续完成复杂任务的复合系统。此类系统的最终行为并不只由基础模型决定，还受到执行环境、工具接口、上下文与记忆、生命周期编排、可观测、验证评估以及治理安全等运行基础设施的共同影响。围绕模型并负责把一次任务组织成可执行过程的这组基础设施，通常被称为 Agent Harness。随着 Agent 任务链条延长，开发者面对的核心困难也从“结果是否正确”进一步转向“为什么形成这一结果，以及下一步应检查哪个机制”。

现有评价工具能够给出任务成功率、基准得分、成本与延迟，却难以把结果直接转化为可操作的系统修改。针对生产环境中 LLM 产品实践者的访谈显示，19 名受访者中有 17 名经历过“结果—可行动性鸿沟”：团队能够收集分数和反馈，却难以判断问题来自提示组织、检索、工具、上下文还是其他系统部件^[1]。这一现象在 Agent 场景中更加突出。一次失败轨迹往往同时包含

自然语言推理、工具调用、环境反馈和中间状态变化，异常可能在早期产生、在后续步骤传播，并最终以前错误结果或提前终止的形式出现。

技术研究已经开始把原始轨迹组织为数据流、控制流和实现锚点，并据此进行自动归因与 Harness 修复。HarnessFix 通过 Harness-aware Trace Intermediate

Representation 把运行步骤与提示模板、工具规范、配置、验证脚本等实现工件对齐，在四类基准上取得 6.3% 至 18.4% 的性能提升^[3]。这些成果说明轨迹—机制关联具有明确的工程价值，但其主要消费者仍是自动诊断与修复模块。自动系统能够生成关联或候选原因，并不等同于开发者能够正确理解这些关系。已有自动故障归因研究中，责任 Agent 的识别准确率约为 53.5%，具体步骤的准确率仅为 14.2%^[6]，表明系统输出本身仍需要核查。

另一方面，Agent 可观测与交互调试工具已能提供时间线、层级视图、断点、回放和现场编辑。AGDebugger、AgentStepper、AgentGUI 等研究证明，结构化轨迹有助于浏览长会话、识别部分缺陷并降低操作负担^[13-14,31]。然而，现有界面常把原始记录、规则推导和模型生成的候选解释放在相近的视觉层级，用户容易把“界面画出的关系”理解为“系统已经证明的因果关系”。例如，工具结果出现在日志中，只能证明该结果产生过；若缺少后续请求快照，就不能断言它已被模型消费。固定结构图中的连线也可能表示预设架构，而非本次运行真实经过的路径。

因此，本研究把问题限定为 Agent Harness 的运行理解：开发者如何把一次运行

中的现象与背后起作用的 Harness 机制联系起来，并知道自己凭什么可以这样联系。

研究进一步聚焦上下文构建与变换过程，考察信息从何处进入模型请求、在后续步骤中如何被保留、替换、裁剪或摘要，以及现有证据能否支持某项故障判断。相较于覆盖全部 Harness 层，这一范围既保留了问题的代表性，也便于构造具有客观真值的研究材料。

3.1.3 研究目的及意义

本研究旨在构建一套面向开发者运行理解的轨迹—机制关联与证据状态呈现方

法。具体而言，首先通过真实运行案例和形成性研究识别开发者在上下文来源追踪、信息变换追踪和故障定位中的证据需求与误判模式；其次在既有 **Architecture** 原型基础上，将数据流、控制流与实现锚点组织为可下钻的关联视图，并把每项关系区分为“实际记录到”“按规则推导”和“目前无法知道”三类证据状态；最后通过具有客观真值的三条件受控实验，检验关联组织与证据状态表达对理解正确率、证据定位、完成时间和无依据结论的影响。

本研究的理论意义主要体现在三个方面。第一，将传统程序调试中的 **why** 与 **why-not** 问题、信息觅食和层级排故研究延伸到由自然语言推理、工具交互与运行时上下文共同构成的 **Agent** 系统，检验既有调试认知理论在概率性运行环境中的适用边界。第二，引入 **Data - Frame** 意义建构框架，解释开发者如何在轨迹证据与机制假设之间循环修正理解，并把“无依据结论”作为运行理解质量的重要表现。第三，把证据状态作为 **Agent** 可观测界面的独立设计变量，讨论记录、推导与未知如何影响使用者对系统结论的校准。

本研究的实践意义在于为 **Agent** 可观测和调试工具提供可检验的交互设计依据。研究不以继续增加日志字段为目标，而是关注已有证据如何被组织、核验和限定。

预期成果包括一组可复用的运行理解任务、一个证据状态显式的轨迹—机制关联原型、一套受控评价方法以及相应设计原则。上述成果可为 **Agent** 开发平台、可观测工具和自动诊断系统的人机接口提供参考，使开发者能够更快定位证据，同时在证据不足时保留“尚不能判断”。

3.2 国内外研究综述

本报告的国内外研究综述见第 2 章“文献综述”，此处不再重复排版。

3.3 研究问题与方法

3.3.1 研究问题

本研究关注一次 Agent 运行已经发生、系统暂不自动修复的情境。核心问题不是简单增加日志数量，而是帮助开发者把分散的运行记录组织为可以检查、可以说明依据且能够保留未知边界的机制理解。基于前述研究现状与不足，提出以下三个研究问题：

RQ1：开发者在理解 Agent Harness 运行过程和定位故障时，需要哪些跨步骤、跨组件的证据关联？现有的原始轨迹与静态结构视图会在哪些任务上导致找不到证据、误解连线或把推测当作事实？

RQ2：在底层运行记录相同的条件下，把分散事件组织为数据流、控制流与实现锚点的轨迹—机制关联，能否提高开发者的运行理解正确率、证据定位准确率并缩短定位时间？ **RQ3：**在轨迹—机制关联已经呈现的基础上，进一步标明“实际记录到”“按规则推导”和“目前无法知道”三类证据状态，能否减少开发者在证据不足时作出确定判断的比例，并在认知负荷可接受的前提下改善结论校准？

3.3.2 研究方法

本研究采用“形成性研究—原型设计—受控评估”的混合研究路径。研究一识别实际困难并形成理解任务集，研究二把困难转化为设计要求并实现原型，研究三通过条件隔离检验界面组织与证据状态表达的作用。三个阶段使用同一组核心概念和可追溯的材料链，保证研究发现能够对应到设计机制，设计机制又能够被实验评价。

3.3.2.1 第一阶段：形成性研究

第一阶段拟招募 8 至 12 名具有 Agent 开发、调试或高频使用经验的参与者。研究材料由预先采集的真实运行记录和少量经过仪器化处理的构造案例组成，重点覆盖三类情境：上下文来源混淆，即某条约束、历史信息或检索材料从何处进入当前请求；压缩、裁剪或替换后的信息变化，即某条内容在后续请求中是否仍然存在；工具结果进入后续请求的过程，即工具输出是否被保存、转换并被模型实际看见。

参与者使用现有的原始轨迹和静态 **Architecture** 视图完成排查任务，并在操作过程中出声思考。研究记录其导航路径、视图切换、证据选择、假设变化、放弃点和最终结论，任务后进行半结构化访谈。分析采用反思性主题分析，并以 **Data - Frame** 理论中的框架形成、扩展、质疑、比较和重构活动作为敏化概念，同时允许新的困难类型从材料中涌现。阶段性产出包括困难清单、理解任务集、证据需求清单和初步设计要求。

为避免把“界面更直观”误当作研究贡献，本阶段设置明确的继续条件：至少识别出能够在多个参与者或多个案例中重复出现、并实际影响答案正确性的困难模式；若主要反馈仅涉及视觉偏好而没有稳定的证据困难，则缩小研究范围或重新构造任务，不直接进入正式原型评估。

3.3.2.2 第二阶段：关联界面原型设计

第二阶段以已有 **Architecture** 原型为基础进行迭代。原型保留固定运行结构、会话一轮次一步骤联动以及上下文构成与运行链路并列三项特征，并新增三类核心机制。

第一，建立轨迹事件与 **Harness** 机制之间的关联。数据流关系用于表示信息的来源、传递、变换与消失；控制流关系用于表示重试、委托、校验、终止等运行决策；实现锚点用于把运行时证据连接到提示模板、工具规范、配置项、上下文处理函数或验证脚本。第二，为每项关联增加证据状态。实际记录到的关系必须能够返回原始事件；按规则推导的关系必须能够展开其输入、规则和适用前提；目前无法知道的关系需说明缺少什么信号以及可能的补证方式。第三，

支持从症状到机制和从系统结构到事件的双向下钻，以兼容症状搜索与拓扑搜索两种排故策略。

原型开发遵循“发现—设计要求—界面机制”的映射。每一项核心交互都需标明其来自研究一的哪类困难，并在研究三中对应至少一个客观指标。原型将先经过专家走查和小规模可用性测试，修正术语理解、状态识别、跨视图选中和返回原始证据等问题，再进入正式实验。

3.3.2.3 第三阶段：三条件受控实验

研究三采用三条件对照设计。所有条件使用相同的运行案例和底层记录，仅改变信息组织与证据状态表达，以避免把“增加了更多数据”和“设计得更好”混为一谈。

实验任务覆盖事件顺序理解、信息来源追踪、上下文变换判断、机制定位和证据充分性判断。为降低学习效应，条件顺序采用拉丁方平衡，正式任务前设置不计分的训练案例；不同条件使用结构等价但内容不同的材料。参与者背景将记录 Agent 开发经验、传统调试经验与可视化使用经验，用于检验个体差异。

3.3.2.4 材料构建与客观真值

运行理解研究必须具备可核对的标准答案，否则只能评价“是否觉得清楚”。本研究采用“真实运行记录加仪器化构造案例”的组合。真实案例用于保留长轨迹、

表 3.1 实验条件设置

条件	呈现方式	主要比较目的
A 常规轨迹	以时间顺序呈现模型请求、工具调用、结果和状态变化	作为现有日志查看方式的基线
B 轨迹—机制关联	在 A 的记录基础上增加数据流、控制流和实现锚点，并支持双向下钻	与 A 比较，检验关联组织的作用

C 关联加证据状态 在 B 的基础上明确标注记录、推导与未知，并提供状态依据 与 B 比较，检验证据状态表达的作用

噪声和非理想数据等生态特征；构造案例用于有计划地操纵上下文保留、替换、裁剪、摘要和工具结果注入等机制，使关键问题具有明确答案。

每个案例均建立独立的证据清单，记录原始事件、稳定标识、关联规则、实现位置和不可观测边界。案例由两名研究者依据统一手册独立标注，并报告评分者间一致性；存在分歧的案例经讨论后修订或删除。对“未知”状态的判定同样需要依据：只有在必需信号确实未记录、采样或超出观测边界时，才标记为目前无法知道。
该过程既保证实验真值，也为原型中的状态说明提供数据基础。

3.3.2.5 测量与数据分析

研究三的主要因变量包括运行理解正确率、证据定位准确率、完成时间和无证据下结论率。运行理解正确率衡量参与者对事件来源、变换机制和故障位置的判断与客观真值是否一致；证据定位准确率衡量其能否找到支撑结论的轨迹片段与实现锚点；无证据下结论率用于统计参与者在证据不足时仍作肯定判断，或把规则推导误称为直接事实的比例。辅助指标包括导航路径、视图切换、答案修改、Raw NASA-TLX 与主观可用性评价。

量化数据在试次层面采用混合效应模型分析，以界面条件和任务类型为固定效应，以参与者和材料为随机效应；二值结果使用广义线性混合模型，连续结果根据分布选择线性混合模型或稳健方法。重点比较 A 与 B、B 与 C，而不是仅报告三组总体差异。样本量由预测试所得效应和随机效应方差通过模拟功效分析确定，不预先沿用旧方案的固定人数。交互日志、屏幕记录和访谈用于解释“界面没有提供线

表 3.2 实验测量维度与指标

测量维度	主要指标	评价重点
------	------	------

理解结果	运行理解正确率、故障定位正确	是否形成与真实机制一致的解釋率
证据能力	证据定位准确率、关键证据召回	是否找到并引用了正确证据率
结论校准	无证据下结论率、未知保留率	是否区分事实、推导与未知
效率与负荷	完成时间、导航步数、Raw NASA-TLX	设计收益是否伴随过高成本

索”“参与者没有发现线索”以及“发现后仍作出错误推断”三类机制差异。

3.3.2.6 可行性与伦理

本研究已具备运行记录、文献基础和 Architecture 原型，可在不搭建在线自动修复平台的条件下开展。材料使用公开框架、自有原型和预先采集的运行记录，不依赖企业内部数据。涉及人的研究将在取得伦理审查批准后实施，参与者均需完成知情同意；数据以研究编号替代身份信息，屏幕记录和访谈转录在去标识后分析，参与者可以随时退出。研究三将在正式收集数据前预注册研究设计、主要指标与分析计划，并在不泄露个人信息的前提下开放任务材料、评分手册和分析代码。

3.4 研究框架

3.5 计划进度

本研究计划以形成性研究、原型设计和受控评估为主线推进。案例真值构建、伦理审查和参与者招募将提前并行开展，以降低后续实验等待成本。各阶段核心任务和预期成果如下。

表 3.3 研究计划与进度安排

阶段	时间	核心任务	阶段成果
----	----	------	------

第一阶段：准备与案例构建	2026.09—2026.10	完成选题报告与伦理申请；核验原型数据边界；采集真实运行记录；建立案例真值与标注手册	材料库 v0；伦理申请；开题答辩
第二阶段：形成性研究	2026.11—2027.01	招募 8—12 名参与者；完成案例排查、出声思考与访谈；归纳困难模式和证据需求	困难清单；理解任务集；设计要求
第三阶段：原型设计与预测试	2027.02—2027.06	迭代轨迹—机制关联与证据状态界面；开展专家走查和可用性预测试；修订实验材料	可交互原型；材料库 v1；预测试报告
第四阶段：受控实验	2027.07—2027.10	完成功效分析与预注册；实施三条件实验；整理交互日志、量表与访谈数据	正式实验数据集；预注册文档
第五阶段：分析与论文撰写	2027.11—2028.02	开展混合效应分析和过程分析；形成设计原则；撰写学位论文与阶段论文	研究结果；设计原则；论文初稿
第六阶段：修改与答辩	2028.03—2028.05	完成论文修改、查重、送审和答辩准备；整理开放材料与原型说明	学位论文定稿；答辩材料；开放材料

Agent Harness运行理解研究框架



图 3.1 研究框架与实施路径

参考文献

[1] VAN DER MADEN W, SADEK M, XIAO Z, et al. Results-Actionability Gap: Understanding How Practitioners Evaluate LLM Products in the Wild[C]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026: 1-17.

[Z]. 2026.

[2] KHANNA R, DODGE J, ANDERSON A, et al. Finding AI’s Faults with AAR/AI:

An Empirical Study[J]. ACM Transactions on Interactive Intelligent Systems, 2022, 12(1): 1:1-1:33.[Z]. 2026.

[3] CHEN M, WANG J, LIU Z, et al. From Failed Trajectories to Reliable LLM Agents:

Diagnosing and Repairing Harness Flaws[EB/OL]. arXiv:2606.06324v2, 2026.[Z]. 2026.

[4] LIN J, LIU S, PAN C, et al. Agentic Harness Engineering:

Observability-Driven Automatic Evolution of Coding-Agent

Harnesses[EB/OL]. arXiv:2604.25850v4, 2026. [Z]. 2026.

[5] AMINI S, BENAJIBA Y, BERNARDIS C, et al. Open Agent

Specification: Enabling Cross-Framework Comparison of AI

Agents[C]//Proceedings of the ACM Conference on AI and Agentic Systems.

2026: 404-418.[Z]. 2026.

[6] ZHANG S, YIN M, ZHANG J, et al. Which Agent Causes Task Failures

and When? On Automated Failure Attribution of LLM Multi-Agent

Systems[C]//Proceedings of Machine Learning Research. 2025, 267: 76583-

76599.[Z]. 2026.

[7] ALSAYYADA, HUANGKY, PALR. AgentTrace: A Structured Logging Fra

mework for Agent System Observability[EB/OL]. arXiv:2602.10133,

2026.[Z]. 2026.

[8] ZHENG Y, HU Y, YU T, et al. AgentSight: System-Level Observability for AI Agents

Using eBPF[EB/OL]. arXiv:2508.02736, 2025.[Z]. 2026.

- [9] KATZ I R, ANDERSON J R. Debugging: An Analysis of Bug-Location Strategies[J]. Human-Computer Interaction, 1987-1988, 3(4): 351-399.[Z]. 2026.
- [10] KO A J, MYERS B A. Designing the Whyline: A Debugging Interface for Asking Questions about Program Behavior[C]//Proceedings of the 2004 ACM Conference on Human Factors in Computing Systems. 2004: 151-158.[Z]. 2026.
- [11] KO A J, MYERS B A. Finding Causes of Program Output with the Java Why-line[C]//Proceedings of the 27th ACM Conference on Human Factors in Computing Systems. 2009: 1569-1578.[Z]. 2026.
- [12] KO A J, MYERS B A. Debugging Reinvented: Asking and Answering Why and Why Not Questions about Program Behavior[C]//Proceedings of the 30th International Conference on Software Engineering. 2008: 301-310.[Z]. 2026.
- [13] EPPERSON W, BANSAL G, DIBIA V, et al. Interactive Debugging and Steering of Multi-Agent AI Systems[C]//Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems. 2025: 156:1-156:15.[Z]. 2026.
- [14] HUTTER R, PRADEL M. AgentStepper: Interactive Debugging of Software Development Agents[EB/OL]. arXiv:2602.06593, 2026.[Z]. 2026.
- [15] LAWRENCE J, BOGART C, BURNETT M, et al. How Programmers Debug, Revis-

ited: An Information Foraging Theory Perspective[J]. IEEE Transactions on Software Engineering, 2013, 39(2): 197-215.[Z]. 2026.

[16] KO A J, MYERS B A, COBLENZ M J, et al. An Exploratory Study of How Devel-

opers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks[J]. IEEE Transactions on Software Engineering, 2006, 32(12): 971-987.[Z]. 2026.

[17] RASMUSSEN J, JENSEN A. Mental Procedures in Real-Life Tasks: A Case Study of Electronic Trouble Shooting[J]. Ergonomics, 1974, 17(3): 293-307.[Z]. 2026.

[18] ROMERO P, DU BOULAY B, COX R, et al. Debugging Strategies and Tactics in a Multi-Representation Software Environment[J]. International Journal of HumanComputer Studies, 2007, 65(12): 992-1009.[Z]. 2026.

[19] CORNELISSEN B, ZAIDMAN A, VAN DEURSEN A. A Controlled Experiment

for Program Comprehension through Trace Visualization[J]. IEEE Transactions on Software Engineering, 2011, 37(3): 341-355.[Z]. 2026.

[20] KLEIN G, MOON B, HOFFMAN R R. Making Sense of Sensemaking 1: Alternative Perspectives[J]. IEEE Intelligent Systems, 2006, 21(4): 70-73.[Z]. 2026.

[21] KLEIN G, MOON B, HOFFMAN R R. Making Sense of Sensemaking 2: A

Macroognitive Model[J]. IEEE Intelligent Systems, 2006, 21(5): 88-92.[Z]. 2026.

- [22] KLEIN G, PHILLIPS J K, RALL E L, et al. A Data-Frame Theory of Sensemak-
ing[M]//Expertise Out of Context. Mahwah, NJ: Lawrence Erlbaum Associates,
2007: 113-155.[Z]. 2026.
- [23] PIROLI P, CARD S K. The Sensemaking Process and Leverage Points
for Analyst Technology as Identified Through Cognitive Task
Analysis[C]//Proceedings of the 2005 International Conference on
Intelligence Analysis. 2005.[Z]. 2026.
- [24] KADIVAR N, CHEN V, DUNSMUIR D, et al. Capturing and
Supporting the Analysis Process[C]//Proceedings of the IEEE Symposium on
Visual Analytics Science and Technology. 2009: 131-138.[Z]. 2026.
- [25] XU K, ATTFIELD S, JANKUN-KELLY T J, et al. Analytic Provenance
for Sensemaking: A Research Agenda[J]. IEEE Computer Graphics and
Applications, 2015, 35(3): 56-64.[Z]. 2026.
- [26] STARR A, STOREY M A. Theory of Troubleshooting: The Developer's
Cognitive Experience of Overcoming Confusion[J]. ACM Transactions on
Software Engineering and Methodology, 2026. DOI: 10.1145/3800945.[Z].
2026.
- [27] FONSECA R, PORTER G, KATZ R H, et al. X-Trace: A Pervasive
Network Tracing
Framework[C]//Proceedings of the 4th USENIX Symposium on Networked
Systems
Design and Implementation. 2007: 271-284.[Z]. 2026.
- [28] SIGELMAN B H, BARROSO L A, BURROWS M, et al. Dapper, a
Large-Scale Dis-

tributed Systems Tracing Infrastructure[R]. Google Technical Report dapper-2010-1, 2010.[Z]. 2026.

[29] MACE J, ROELKE R, FONSECA R. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems[C]//Proceedings of the 25th ACM Symposium on Operating Systems Principles. 2015: 378-393.[Z]. 2026.

[30] LUJ,PANB,CHENJ,etal.AgentLens:VisualAnalysisforAgentBehaviorsinLLMBased Autonomous Systems[J]. IEEE Transactions on Visualization and Computer Graphics, 2025, 31(8): 4182-4197.[Z]. 2026.

[31] ZHAO X, SOHN J, ZHENG Q, et al. AgentGUI: An Interface for Observing and

Steering Long-Running AI Agents[EB/OL]. arXiv:2607.26300, 2026.[Z]. 2026.

[32] GAO T, LI H, LI J H, et al. Graph of Trace: Visualizing Execution Traces of Scientific Agents[C]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. 2026: 297-306.[Z]. 2026.

[33] WU Z, CHO S, VILLALOBOS C E M, et al. AgentGraph: Trace-to-Graph Platform for Interactive Analysis and Robustness Testing in Agentic AI Systems[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2026, 40(48): 41721-41723.[Z]. 2026.

[34] DIBIA V, CHEN J, BANSAL G, et al. AutoGen Studio: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 2024: 72-79.[Z]. 2026.

[35] RORSETHJ,GODFREYP,GOLABL,etal.LADYBUG:anLLMAgentDeBUGger

for Data-Driven Applications[C]//Proceedings of the 28th International Conference on Extending Database Technology. 2025: 1082-1085.[Z]. 2026.

[36] PARNIN C, ORSO A. Are Automated Debugging Techniques Actually Helping

Programmers?[C]//Proceedingsofthe20thInternationalSymposiumonSoftware Testing and Analysis. 2011: 199-209.[Z]. 2026.

[37] RAGAN E D, ENDERT A, SANYAL J, et al. Characterizing Provenance in Visualization and Data Analysis: An Organizational Framework of

Provenance Types and

Purposes[J].IEEETransactionsonVisualizationandComputerGraphics,2016,22(1): 31-40.[Z]. 2026.

[38] WANG Y, ZHANG J, CAI T, et al. From Agent Traces to Trust: Evidence Tracing and

Execution Provenance in LLM Agents[EB/OL]. arXiv:2606.04990, 2026.[Z]. 2026.

[39] SACHA D, SENARATNE H, KWON B C, et al. The Role of

Uncertainty, Awareness, and Trust in Visual Analytics[J]. IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 240-249.[Z]. 2026.

[40] HULLMAN J. Why Authors Don't Visualize Uncertainty[J]. IEEE Transactions on

Visualization and Computer Graphics, 2020, 26(1): 130-139.[Z]. 2026.

华中科技大学

研究生（文献阅读与选题）报告

☐文献阅读、☐选题、☐文献阅读与选题、☐论文中期进展

题目：AI Agent 运行轨迹与机制关联的交互设计研究

学号 M202570493

姓名 陈思静

专业 设计学

指导教师 何心

院（系、所） 设计学院